

Article

# Reconstructing CNC G-Code from Multi-Modal Sensor Embeddings: An Autoregressive Multi-Head Decoder with Grammar Constraints

Stephen S. Eacuella <sup>1,\*</sup> , Romesh S. Prasad <sup>1</sup> and Manbir S. Sodhi <sup>1</sup>

<sup>1</sup> Department of Industrial Systems Engineering, University of Rhode Island, Kingston, RI 02881, USA

\* Correspondence: seacuella@uri.edu

Version April 13, 2026 submitted to Sensors

**Abstract:** A CNC machine’s sensor streams encode far more than tool health or process stability—they encode the physical execution of the program itself. This paper studies how much of a G-code line can be read back from multi-modal sensor embeddings of the machining it produced. On a controlled desktop milling dataset (120 files, 545 windows, nine operation classes, single material), an autoregressive Transformer decoder conditioned on a frozen multi-modal encoder recovers the executed G-code line with **84.5%** mean cross-fold sequence accuracy (single seed; best-of-5-seed retrain reaches 86.1%) and 98.1% motion-command accuracy. Two findings are central. First, G-code reconstruction has a physical ceiling: successive lateral step-over passes produce sensor dynamics that are statistically indistinguishable within one observation window, so numeric coordinate prediction requires context the sensors cannot provide. Quantitatively, a model given only sensor embeddings (no within-file position metadata) reaches 62.3% sequence accuracy; adding a single ordinal position feature lifts this to 86.1%—a 23.8 percentage-point gap that *measures* how much of a program is recoverable from physics alone versus from file structure. On the same data a position-based majority lookup table reaches 90.4% without any model, confirming that on a fixed geometry, position dominates. The decoder’s value is therefore not raw accuracy on the training geometry but rather what it preserves under distribution shift: in held-out-toolpath experiments, command and parameter-type accuracy transfer substantially while both lookup and decoder sequence accuracy collapse to zero, because unseen numeric values cannot be recovered from either method. Second, the modality ranking for reconstruction diverges sharply from the modality ranking for operation classification, with inertial sensors (gyroscope, accelerometer) and the color channel dominating reconstruction while audio—the second-strongest classifier feature—is the least informative. Reconstruction remains viable without the barometrically confounded proximity/pressure channels (82.5%), at half the observation window (76.9%), and with a 3.0M-parameter decoder (84.4%). The resulting structured error patterns—concentrated on Y-axis step-over values—delineate exactly what sensor-based process verification can and cannot detect, a question that motivates but is validated only in a separate companion paper on physical-layer anomaly detection.

**Keywords:** CNC machining; G-code reconstruction; sensor fusion; autoregressive decoding; grammar constraints; process verification; physical-layer monitoring

## 1. Introduction

Modern CNC manufacturing relies on G-code programs to specify toolpaths with sub-millimeter precision. Process verification—confirming that the commanded operation actually occurred—is critical for quality assurance, regulatory compliance, and cybersecurity [1]. Classification-based

monitoring can identify *which* operation type was executed [2,3]. It cannot verify *what specific* G-code was run. A classifier that reports “adaptive clearing” provides no information about whether feed rates, depths, or arc radii match the intended program.

G-code reconstruction from sensor data addresses this gap. Given a window of multi-modal sensor measurements recorded during machining, the goal is to recover the exact G-code line that produced those measurements. This requires predicting three components: the motion command (G0, G1, G2, G3), the parameter addresses (X, Y, Z, F, R), and the numeric coordinate values to machining precision. The task is structurally similar to automatic speech recognition (ASR) [4]. An encoder maps raw signals to learned representations; a decoder generates structured output in the target language. G-code reconstruction differs in that output sequences are short (5–8 tokens) and drawn from a formal grammar rather than open-ended natural language.

The decoder builds on two preceding contributions. The MM-DTAE-LSTM encoder [1] classifies nine CNC operation classes at up to 96.3% accuracy using cross-modal attention, denoising reconstruction, and LSTM temporal modeling. It produces 256-dimensional fingerprint embeddings that capture toolpath-discriminative information. A formal G-code grammar and hierarchical tokenization scheme [5] showed that G-code toolpath syntax is a regular language with only 19 structural tokens. Numeric coordinates are best predicted digit-by-digit rather than as monolithic vocabulary entries.

These foundations combine into a sensor-to-G-code pipeline. The encoder is frozen: its pretrained weights produce sensor embeddings that serve as cross-attention memory for the decoder. The decoder is an autoregressive Transformer that generates G-code tokens one at a time. A multi-head loss decomposes prediction into token type, command, parameter, and digit-level components. Grammar constraints enforce syntactic validity via logit masking at each decoding step.

Reconstruction is harder than classification by an order of magnitude in output complexity. Classification selects from 9 operation types. Reconstruction must predict exact G-code sequences from the 712-token vocabulary, including numeric coordinates that differ by fractions of a millimeter.

Four research questions guide this work:

- RQ1.** Can exact G-code sequences be reconstructed from sensor embeddings, and at what accuracy?
- RQ2.** Does architectural complexity or data-handling correctness drive reconstruction accuracy?
- RQ3.** Which sensor modalities matter most for reconstruction, and does the importance ranking differ from classification?
- RQ4.** What is the minimum sensor set, observation window, and decoder capacity for viable reconstruction?

The contributions are as follows:

1. **Sensor-to-G-code reconstruction as a benchmark task (RQ1).** To our knowledge this is the first system that reconstructs the exact token sequence of a CNC G-code line from multi-modal sensor embeddings on a controlled desktop milling dataset. It achieves 84.5% mean cross-fold sequence accuracy (86.1% with best-of-5-seeds selection), 94.9% token accuracy, and 98.1% command accuracy across five folds (Section 7).
2. **SensorMultiHeadDecoder architecture.** A Transformer decoder with hierarchical multi-head prediction, grammar-constrained decoding, multi-window temporal context, and an ordinal window-position feature for structured sequence generation from sensor embeddings (Sections 4–5).
3. **A physics ceiling on numeric coordinate reconstruction (RQ2).** A position-based majority lookup table achieves 90.4% cross-fold sequence accuracy on this dataset without any model—higher than any learned baseline—which motivates asking what the sensors themselves can actually recover. Removing the ordinal window-position feature from the full decoder drops accuracy from 86.1% to 62.3%, a 23.8 pp gap that directly quantifies how much of a G-code line is recoverable from sensor physics alone versus from file-structure metadata. The drop is concentrated on Y-axis step-over coordinates, where successive lateral passes produce sensor dynamics that are statistically

indistinguishable within one observation window—a physical, not representational, limitation. No architectural feature of the decoder moves accuracy by more than 2.1 pp (Section 7.2).

4. **Held-out-toolpath evaluation (new experiment).** To test whether the decoder generalizes beyond memorized file structures, we hold out an entire toolpath class at training time and evaluate on the held-out toolpath. On these splits the lookup table collapses to 0% on all metrics while the decoder preserves command and parameter-type accuracy at levels substantially above the lookup table—although both methods still produce 0% exact-sequence accuracy because the held-out toolpath uses numeric coordinate values the model has never seen (Section 7.5).
5. **Sensor modality analysis (RQ3).** A 9-modality ablation reveals that the sensor importance ranking for reconstruction diverges from classification (Spearman  $\rho = -0.35$ ). Gyroscope (−27.7 pp) and color (−27.2 pp) dominate reconstruction, while audio—ranked #2 for classification—is least important (−6.5 pp) (Section 7.3.1). We explicitly tested the hypothesis that the color sensor correlates with Y-axis lateral position via ambient-light gradients and found no meaningful correlation; the mechanism underlying the color channel’s reconstruction importance remains unexplained.
6. **Minimum viable configuration (RQ4).** Reconstruction remains viable without the barometrically confounded proximity/pressure channels (82.5%), at half the observation window (76.9%), and with a 3.0M-parameter decoder (84.4%) (Section 7.4.1).

This paper is deliberately scoped as a proof-of-concept study on a single machine, single material, and single tool, establishing what can and cannot be read from sensor data under controlled conditions. Validation of the anomaly-detection application suggested by the structured error patterns (Section 9) is the subject of a separate companion paper [6] (in preparation as of this writing) and is not claimed here.

The remainder of this paper is organized as follows. Section 2 reviews related work. Section 3 formulates the reconstruction task. Sections 4–5 describe the decoder architecture and temporal context features. Section 6 details the experimental setup. Section 7 presents results addressing RQ1–RQ4 in order. Section 8 analyzes error patterns. Section 9 discusses implications, limitations, and future directions.

## 2. Related Work

This section reviews prior work in four areas that inform the present study. Section 2.1 surveys CNC process monitoring. Section 2.2 covers sequence-to-sequence models applied to manufacturing. Section 2.3 addresses G-code tokenization. Section 2.4 discusses manufacturing cybersecurity and positions the reconstruction task.

### 2.1. CNC Process Monitoring and Sensor Fusion

Sensor-based CNC monitoring has focused on tool condition monitoring [2,7], chatter detection [8], surface quality prediction [9], and fault classification [3]. Recent deep learning approaches include CNNs for raw CNC signal classification [10] and Transformer-based multi-sensor fusion for tool wear [11]. Broader reviews of machine learning for production-process optimization [12] document many such classification pipelines. All these approaches classify sensor data into discrete categories (healthy/worn, stable/chattering) rather than reconstructing the underlying program.

The MM-DTAE-LSTM encoder [1] advanced this paradigm to multi-class operation classification across nine CNC operation types with systematic sensor ablation. However, reconstruction of the actual G-code—recovering the specific commands and numeric coordinates—remained as future work. No prior study in CNC monitoring has attempted to recover the executed program from sensor data.

### 2.2. Sequence-to-Sequence Models for Manufacturing

Sequence-to-sequence architectures [13,14] are standard for structured-output generation in speech and language tasks; their application to manufacturing programs, however, is recent. Table 1

compares recent work involving G-code and sensor-based manufacturing models. Recent efforts apply large language models to G-code in the generation direction: GLLM [15] fine-tunes a code LLM for natural-language-to-G-code translation, Image2Gcode [16] generates toolpaths from images via diffusion, ChatCNC [17] enables conversational machine monitoring, and foundational G-code models [18] explore program comprehension and manipulation.

All these works operate in the program generation or comprehension direction (text/image→G-code or G-code→text). None operate in the reconstruction direction (sensors→G-code). The present work inverts the information flow: given what the machine *did* (sensor data), recover what it was *told to do* (G-code).

**Table 1.** Comparison with related work involving G-code or sensor-based manufacturing models. No prior work reconstructs G-code from sensor data. Evaluation metrics differ across studies and are not directly comparable.

| Study              | Direction             | Input               | Output              | Method             | Eval                                 |
|--------------------|-----------------------|---------------------|---------------------|--------------------|--------------------------------------|
| GLLM [15]          | Text→G-code           | Natural language    | G-code program      | Fine-tuned LLM     | Qualitative                          |
| Image2Gcode [16]   | Image→G-code          | 3D image            | Toolpath            | Diffusion          | Geom. error                          |
| ChatCNC [17]       | G-code→Text           | Machine data        | Conversational      | LLM + RAG          | User study                           |
| Found. G-code [18] | G-code↔Text           | G-code              | Comprehension       | LLM                | Accuracy                             |
| Encoder [1]        | Sensors→Class         | 110 channels        | 9 op. classes       | MM-DTAE-LSTM       | 96.3% (5-fold)                       |
| <b>Ours</b>        | <b>Sensors→G-code</b> | <b>110 channels</b> | <b>Exact G-code</b> | <b>Transformer</b> | <b>84.5% (5-fold, mean-of-seeds)</b> |

### 2.3. G-Code Tokenization and Grammar

Standard subword tokenizers fragment G-code destructively [5]. BPE splits X3.275 into semantically meaningless fragments, and vocabulary scales with part geometry. The hybrid tokenization from [5] separates structural tokens (19 total: 5 special, 4 motion commands, 5 parameters, 5 auxiliary literals) from numeric values. For numeric prediction, the framework decomposes each coordinate into a sign token (positive/negative/zero) and six individual digit tokens (one per decimal position), producing a fixed-size representation regardless of the coordinate value. This digit-by-digit approach means a coordinate like X3.275 is predicted as the sequence [sign=+,  $d_1=3, d_2=2, d_3=7, d_4=5, d_5=0, d_6=0$ ] rather than as a single monolithic vocabulary entry, decoupling the vocabulary from part geometry. Grammar-constrained decoding via logit masking [19–21] guarantees syntactically valid G-code at  $O(1)$  cost per step. This tokenization framework is adopted throughout this work.

### 2.4. Manufacturing Cybersecurity

CNC systems are vulnerable to G-code injection, parameter modification, and controller compromise [22,23]. The NIST FMNet roadmap [24] identifies cyberphysical verification of machining processes as a critical research need. Sturm *et al.* [25] demonstrated that subtle G-code modifications in additive manufacturing can introduce internal defects invisible to visual inspection. Yampolskiy *et al.* [26] surveyed attack vectors across digital manufacturing supply chains. Kimmell *et al.* [27] showed that energy data anomalies can detect control injection attacks in CNC machining.

Existing defenses rely on network monitoring, controller log analysis, or threshold alarms on individual sensor channels, and recent work uses energy-data anomalies to flag control-injection attacks in CNC machining [27]. Sensor-based reconstruction offers an orthogonal approach. By independently predicting what G-code *should have been* executed from physical measurements, deviations between predicted and commanded G-code reveal tampering that may be invisible to the controller itself. This security application motivates the reconstruction task.

### 3. Problem Formulation

#### 3.1. Task Definition

Given a window of  $W$  timesteps of multi-modal sensor data  $\mathbf{X} \in \mathbb{R}^{W \times F}$  (where  $F$  is the number of sensor features) recorded during CNC machining, reconstruct the corresponding G-code token sequence  $\mathbf{y} = (y_1, y_2, \dots, y_T)$  where each  $y_t$  belongs to a vocabulary  $\mathcal{V}$  of 712 tokens.

#### 3.2. Vocabulary and Token Types

Following [5], the vocabulary decomposes into four primary types:

- **SPECIAL** (5): PAD, BOS, EOS, UNK, MASK
- **COMMAND** (6): G0, G1, G2, G3, G53, M30
- **PARAMETER** (5): X, Y, Z, F, R
- **NUMERIC** (693): axis-specific coordinate tokens, *e.g.* NUM\_X\_3275, where the suffix is the coordinate value in units of 0.0001 of the axis unit (four decimal places)
- **LEGACY** (3): full-value tokens retained from earlier vocabulary versions

The dataset contains 34 unique G-code target sequences across 545 sensor windows (27 per fold after stratified splitting). Of these, 17 sequences have fewer than 20 windows each, creating a long-tail distribution. A separate audit of how many vocabulary entries are actually exercised by the training data shows that only 40 of the 693 numeric tokens—and only 14 of the structural tokens—appear in any target sequence, so the effective output space is 48 tokens even though the nominal vocabulary is 712.

#### 3.3. Evaluation Metrics

Evaluation uses multiple granularities:

- **Sequence accuracy**: Fraction of windows where the entire predicted G-code sequence exactly matches the target.
- **Token accuracy**: Per-position token match rate.
- **Command accuracy**: Accuracy on command tokens (G0/G1/G2/G3) only.
- **Numeric accuracy**: Accuracy on numeric coordinate tokens only.
- **Parameter type accuracy**: Accuracy on parameter address tokens (X/Y/Z/F/R).

All reported accuracies use teacher-forced evaluation: at each decoding step, the model receives the ground-truth previous token as input and predicts the next token. This is standard practice for autoregressive models during evaluation [14] and matches the training regime (with scheduled sampling bridging the distribution gap; Section 4).

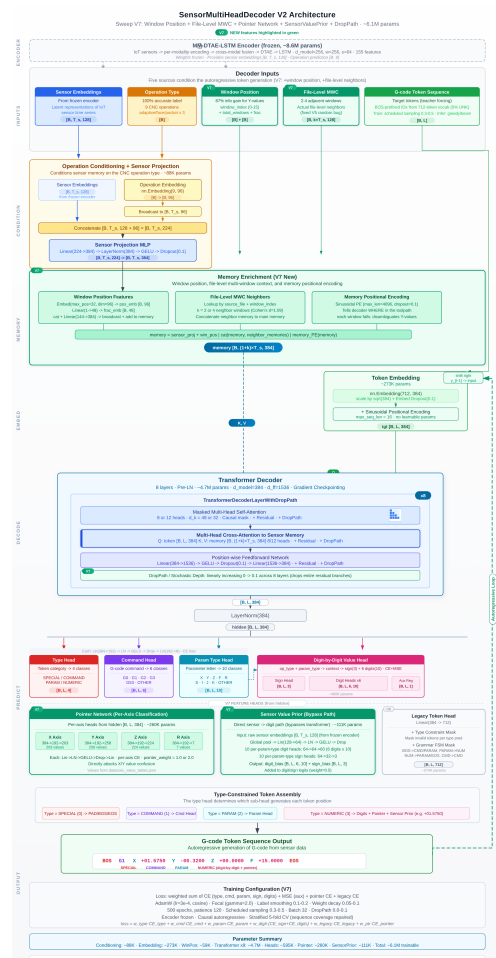
## 4. Architecture

#### 4.1. System Overview

The reconstruction pipeline has two stages (Figure 1):

1. **Encoder** (frozen): The pretrained MM-DTAE-LSTM [1] processes a sensor window  $\mathbf{X} \in \mathbb{R}^{256 \times 110}$  through modality-specific projections, cross-modal attention gates, a denoising temporal attention encoder (DTAE), and a bidirectional LSTM. This produces memory  $\mathbf{M} \in \mathbb{R}^{L \times 256}$  where  $L$  is the compressed sequence length.
2. **Decoder** (trained): The SensorMultiHeadDecoder autoregressively generates G-code tokens conditioned on  $\mathbf{M}$ , using cross-attention at each Transformer layer.

The encoder ( $\sim 5.1\text{M}$  parameters; see [1] for architecture details and parameter breakdown) is frozen throughout all experiments. This design choice is motivated by two factors: (1) with only 545 training samples, fine-tuning a 5.1M-parameter encoder risks catastrophic overfitting; and (2) the frozen encoder serves as a fixed feature extractor, allowing decoder ablations to isolate decoder-specific effects without confounding encoder changes.



**Figure 1.** System architecture. The frozen encoder (left) produces 256-dimensional memory from multi-modal sensor windows. The decoder (right) generates G-code tokens autoregressively via cross-attention, with hierarchical heads for type, command, parameter, and digit prediction.

#### 4.2. Decoder Architecture

The SensorMultiHeadDecoder is a Transformer decoder [14] with  $d_{\text{model}} = 384$ ,  $n_{\text{layers}} \in \{6, 8\}$ , and  $n_{\text{heads}} \in \{8, 12\}$ . Each layer contains:

1. Masked self-attention over previously generated tokens
2. Cross-attention over encoder memory  $\mathbf{M}$
3. Position-wise feed-forward network

The encoder output ( $d = 256$ ) is projected to the decoder dimension ( $d = 384$ ) via a linear layer before cross-attention. The full decoder has 22.1M parameters, of which 85.8% reside in the 8-layer Transformer (self-attention + cross-attention + feed-forward). Notably, the capacity ablations (Section 7.2.2) show that a 2-layer,  $d=192$  variant with 3.0M parameters achieves 84.4% sequence accuracy, suggesting the architecture is overparameterized for this task.

### 4.3. Multi-Head Prediction

Following the hierarchical token decomposition [5], prediction is decomposed into specialized heads:

- **Token type head:** 4-class classifier (Special, Command, Parameter, Numeric)
- **Command head:** 6-class classifier (G0, G1, G2, G3, G53, M30)
- **Parameter type head:** 9-class classifier (X, Y, Z, F, R, S, I, J, K)
- **Digit heads:** Per-position classifiers for sign (3-class) and 6 digits (10-class each)
- **Legacy token head:** Full 712-class vocabulary classifier

The legacy head provides a flat prediction baseline; the hierarchical heads provide structured decomposition. Both contribute to the training loss:

$$\mathcal{L} = \mathcal{L}_{\text{legacy}} + \mathcal{L}_{\text{type}} + \mathcal{L}_{\text{cmd}} + \mathcal{L}_{\text{param}} + w_d \sum_i \mathcal{L}_{d_i} \quad (1)$$

All structural terms ( $\mathcal{L}_{\text{legacy}}$ ,  $\mathcal{L}_{\text{type}}$ ,  $\mathcal{L}_{\text{cmd}}$ ,  $\mathcal{L}_{\text{param}}$ ) are weighted at 1.0; only the digit weight  $w_d$  is tuned, because digit prediction spans six positions and is the dominant source of numeric error.  $w_d \in \{15, 20, 30\}$  was swept in V7; all three values produced identical best accuracy (Section 7.4). Each loss uses focal cross-entropy [28] with  $\gamma = 2.0$  to handle class imbalance; class weights are uniform across heads. Label smoothing ( $\alpha \in \{0.1, 0.2\}$ ) provides additional regularization.

### 4.4. Grammar-Constrained Decoding

At each generation step, a type-transition mask  $M(\tau_{t-1}) \in \{0, -\infty\}^{|V|}$  is applied to the legacy head logits based on the previous token’s type [5]:

$$\tilde{\ell}^{(t)} = \ell^{(t)} + M(\tau_{t-1}) \quad (2)$$

For example, after a PARAMETER token, only NUMERIC tokens are valid; after NUMERIC, only PARAMETER or EOS are valid. Command-specific rules further constrain parameter selection (*e.g.*, in this dataset’s controller dialect, G0 rapid traverse does not use F; G2/G3 circular interpolation requires R for the radius-format arcs used here). These constraints guarantee syntactically valid output regardless of model confidence.

An important finding from our experiments (Section 7.2.2) is that grammar constraints’ effectiveness depends on rule correctness. Initial rules that prohibited command-less sequences (where modal persistence makes the command implicit) *hurt* performance by penalizing valid G-code. After fixing the rules to allow BOS→PARAMETER and NUMERIC→COMMAND transitions, grammar constraints became beneficial for top configurations.

### 4.5. Scheduled Sampling

During training, the decoder uses teacher forcing with scheduled sampling [29]: at each step, the ground-truth previous token is replaced with the model’s own prediction with probability  $p_{ss} \cdot \min(1, \text{epoch}/E)$ , where  $p_{ss} \in \{0.3, 0.5\}$  and  $E$  is the total epochs. This bridges the train-test distribution gap for autoregressive models.

The base decoder described above processes one window at a time. The following section introduces features that extend its context beyond the individual window.

## 5. Temporal Context Features

The single most impactful architectural contribution is providing the decoder with temporal context beyond the individual sensor window.

### 5.1. Memory Positional Encoding

Sinusoidal positional encoding is added to the encoder memory  $\mathbf{M}$  before cross-attention, telling the decoder *where within the compressed sequence* each memory position falls. This is distinct from the standard decoder positional encoding (which tracks output token position) and provides spatial structure to the sensor representation.

### 5.2. Multi-Window Context (MWC)

A single 256-timestep window captures 64 seconds of machining at the  $\sim 4$  Hz aligned sampling rate. Three sampling regimes coexist in the data pipeline: the native LSM9DS1 IMU samples at 119 Hz (59.5 Hz Nyquist), the firmware-level RMS audio and environmental sensors sample at 10 Hz, and the final aligned rate after interpolation to the machine-status timeline is  $\sim 4$  Hz. The 400 Hz tooth-passing frequency exceeds every one of these Nyquist limits and is not recoverable from any of the recorded streams. The question is therefore not whether tooth-passing harmonics survive—they do not—but whether any signal that *does* survive carries toolpath-discriminative information. The encoder paper [1] argues on physical grounds that discriminative content lives in low-frequency structural-mode vibration (1–50 Hz), acceleration transients at toolpath direction changes, and operation-specific trajectory envelopes—frequencies that are either captured natively at 119 Hz or, for trajectory envelopes, present at any rate above the motion bandwidth ( $\sim 1$  Hz). Figure 2 shows empirical power spectral densities at the 4 Hz aligned rate for three representative toolpaths and confirms that meaningful spectral structure exists in the DC–2 Hz band: face milling shows periodic structure corresponding to step-over passes, pocket machining shows burst-like low-frequency energy, and adaptive clearing shows broadband activity consistent with its continuous direction reversals. The 4 Hz aligned rate is thus a deliberate choice that preserves trajectory-envelope information at the cost of high-frequency cutting dynamics—a trade we accept for this work but that would need revisiting for applications targeting sub-second fault detection or chatter characterization.

Many G-code distinctions—particularly Y-axis coordinate values that differ between successive passes—require context spanning multiple windows. Multi-window context concatenates encoder memories from adjacent windows within the same source file:

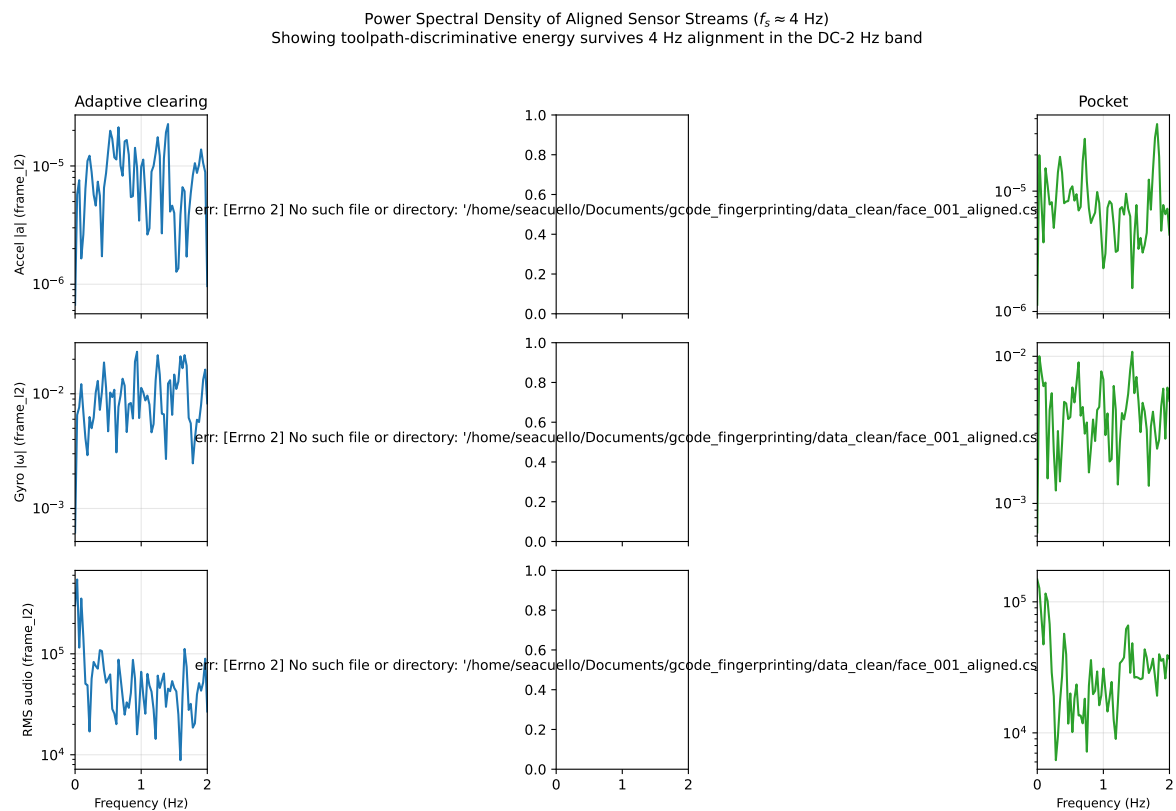
$$\mathbf{M}_{\text{ctx}} = [\mathbf{M}_{i-k}, \dots, \mathbf{M}_i, \dots, \mathbf{M}_{i+k}] \quad (3)$$

where  $k \in \{1, 2\}$  is the context radius. Critically, neighbors must come from the *same source file* (same physical operation recording) and be ordered by window index within that file. Early implementations (V5–V6) used index-based neighbors after DataLoader shuffle, inadvertently producing random windows from unrelated files—a bug that nonetheless showed positive signal (V5 Cohen’s  $d = 1.09$  for the combined MWC + memory positional encoding effect, measured with the buggy random neighbors). Correcting to file-level neighbors in V7 reduced the MWC-specific contribution to  $-2.8$  pp (Table 9), indicating that much of the V5 effect was attributable to window position metadata rather than temporal context per se.

### 5.3. Window Position Embedding

Each window is identified by its position within the source file: (window\_index, total\_windows). A discrete embedding for the window index and a continuous projection of the normalized position (index/total) are concatenated and added to the decoder’s initial hidden state.

Window position provides 57.8% normalized information gain for Y-axis step-over value disambiguation, computed as the reduction in conditional entropy  $H(Y|\text{position})$  relative to the marginal entropy  $H(Y)$  over all training windows containing Y-axis coordinates. The relationship is structured but not deterministic:  $Y = 0.1755$  is the most common Y-value at positions 0–1 (34–38% of windows at those positions), and  $Y = 2.9132$  is dominant at position 2 (52–93% depending on fold). The partial correlation reflects the diversity of toolpath strategies and operating conditions in the dataset:



**Figure 2.** Power spectral density of three representative sensor channels (accelerometer magnitude, gyroscope magnitude, RMS audio from the frame\_12 board) for three representative toolpath files at the 4 Hz aligned sampling rate. Distinct spectral signatures survive the alignment despite the loss of the 400 Hz tooth-passing frequency and 59.5 Hz IMU Nyquist.

each strategy uses different Y-coordinate progressions. Nonetheless, the 57.8% entropy reduction confirms that window position provides substantial disambiguation. This positional information is encoded in the file structure but invisible to a single-window encoder—the sensor dynamics during each pass are governed by cutting conditions (depth, feed, engagement), not the table’s lateral position. Window position was the single highest-impact feature, blocked in V5–V6 due to missing metadata in the preprocessed NPZ files, and enabled in V7 after re-processing the full data pipeline from raw sensor CSVs.

#### 5.4. Pointer Network for Per-Axis Classification

A pointer network was also tested with per-axis classification heads (X: 203 values, Y: 258, Z: 224, R: 7) to replace the flat 712-class legacy head. Each head is a small MLP predicting only the values relevant to the current axis. This feature showed no benefit in V7 (Table A6) and was locked OFF, suggesting that the legacy head’s shared representation is sufficient once the preprocessing is correct.

## 6. Experimental Setup

### 6.1. Dataset

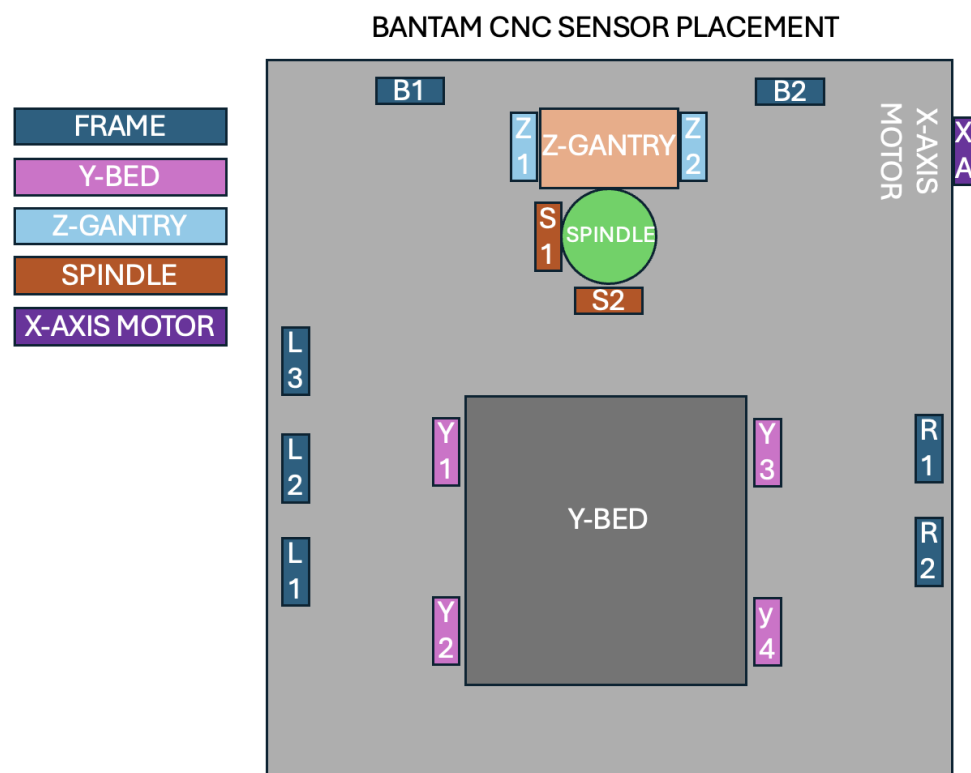
The dataset is the same as [1]: 120 cleaned operation files from a Bantam Tools Explorer™ desktop CNC milling machine. The machine is instrumented with six Arduino Nano 33 BLE Sense Lite sensor boards (Table 2) spanning three structural locations—frame (3 boards), spindle (1), and bed (2), shown in Figure 3—plus machine-level electrical monitoring via MCCDAQ. Each board integrates a 9-axis IMU (LSM9DS1: accelerometer, gyroscope, magnetometer at 119 Hz native), environmental sensors (LPS22HB pressure/temperature, APDS-9960 proximity at 10 Hz), and a PDM microphone computing RMS audio at the firmware level. All streams are aggregated via USB serial on a Raspberry Pi 4, then interpolated and aligned to the machine status timeline at an effective rate of ~4 Hz for G-code labeling. Full instrumentation details, sensor placement photographs, and data cleaning procedures are described in [1].

**Table 2.** Sensor Configuration (6 Boards  $\times$  17 Channels + 8 Electrical = 110 Features)

| Modality                                         | Channels   | Per Board | $\times 6$ |
|--------------------------------------------------|------------|-----------|------------|
| Accelerometer                                    | Ax, Ay, Az | 3         | 18         |
| Gyroscope                                        | Gx, Gy, Gz | 3         | 18         |
| Magnetometer                                     | Mx, My, Mz | 3         | 18         |
| Pressure                                         | Barometric | 1         | 6          |
| Temperature                                      | Ambient    | 1         | 6          |
| Proximity                                        | APDS-9960  | 1         | 6          |
| Color                                            | R, G, B, A | 4         | 24         |
| RMS Audio                                        | PDM mic    | 1         | 6          |
| + Machine electrical (voltage, current, spindle) |            |           | 8          |
| <b>Total</b>                                     |            |           | <b>110</b> |

All operations used a 2-flute HSS flat endmill (1/4" / 6.35 mm diameter) at 12,000 RPM (tooth-passing frequency  $f_t = 400$  Hz) and 1,016 mm/min feed rate. Face milling and pocketing used a 3.81 mm stepover (60% of tool diameter), while adaptive clearing used trochoidal toolpaths with 10–25% radial engagement. These are typical parameters for hobby-grade desktop CNC milling. Active-cutting trials machined TIVAR UHMW-PE (ultra-high molecular weight polyethylene) at 0.635 mm axial depth. This soft polymer was chosen for reproducible, low-variation cutting forces; metals would produce fundamentally different force, thermal, and acoustic signatures.

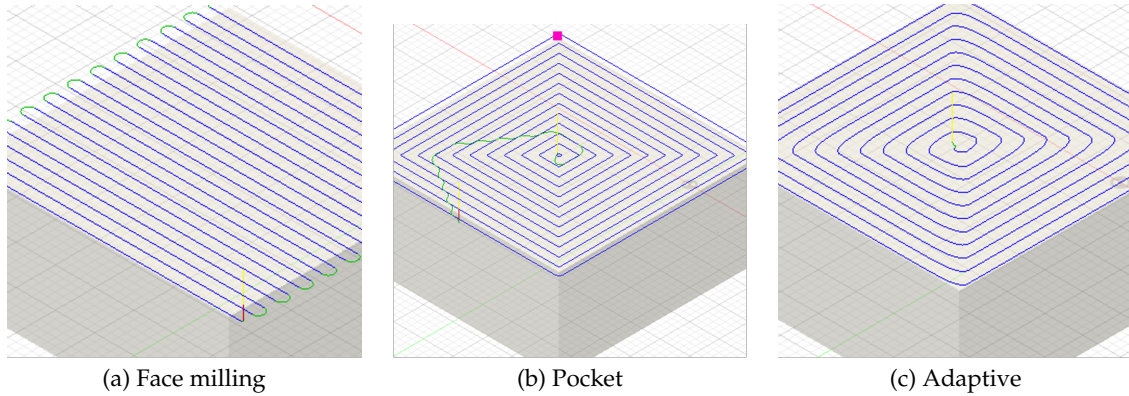
The dataset spans 9 operation classes (Table 3): three toolpath strategies (Figure 4)  $\times$  three operating conditions.



**Figure 3.** Sensor placement schematic showing six Arduino boards distributed across the machine frame (3), spindle (1), and bed (2), providing spatial diversity across the primary vibration transmission paths.

**Table 3.** Nine Operation Classes (3 Toolpaths  $\times$  3 Conditions)

| Toolpath          | Condition  | Label          | Files      |
|-------------------|------------|----------------|------------|
| Adaptive clearing | Air-cut    | adaptive       | 20         |
| Adaptive clearing | Active-cut | adaptive150025 | 17         |
| Adaptive clearing | Damaged    | damageadaptive | 5          |
| Face milling      | Air-cut    | face           | 16         |
| Face milling      | Active-cut | face150025     | 15         |
| Face milling      | Damaged    | damageface     | 4          |
| Pocket machining  | Air-cut    | pocket         | 19         |
| Pocket machining  | Active-cut | pocket150025   | 19         |
| Pocket machining  | Damaged    | damagepocket   | 5          |
| <b>Total</b>      |            |                | <b>120</b> |

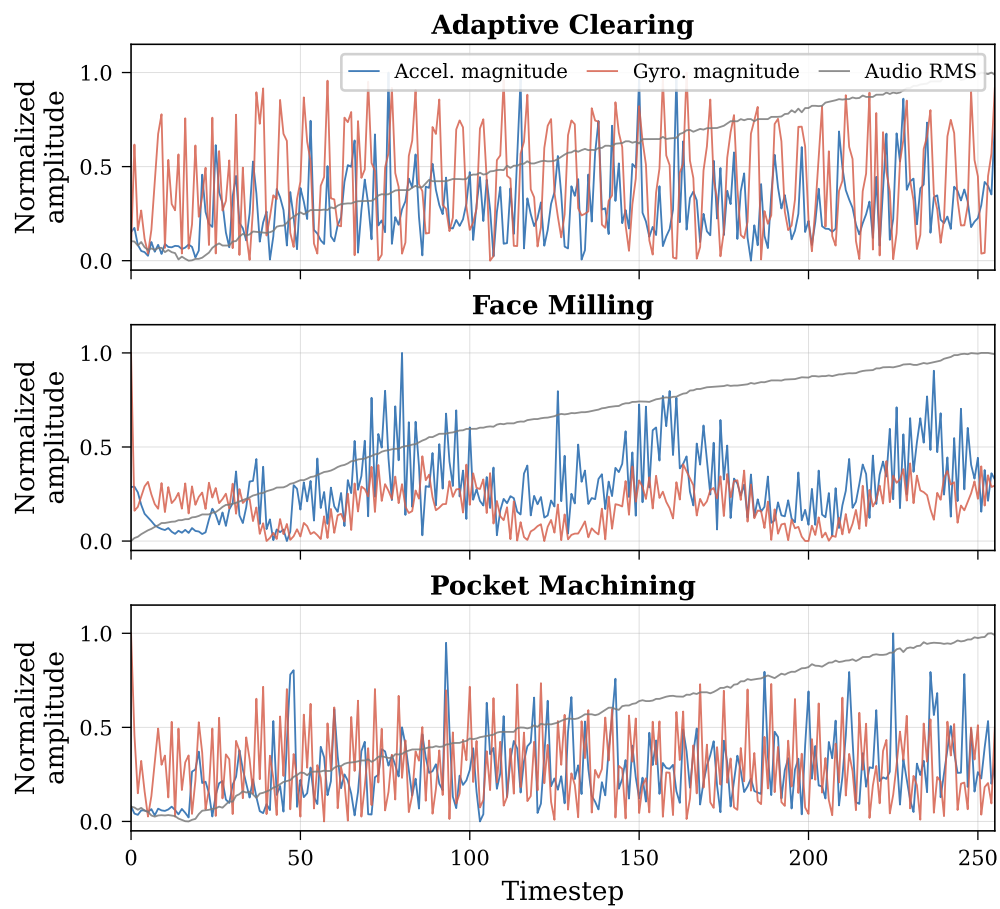


**Figure 4.** CAM toolpath strategies: (a) face milling sweeps in parallel linear passes with periodic step-over transitions; (b) pocket machining alternates between axial plunges and lateral clearing passes; (c) adaptive clearing follows trochoidal paths maintaining near-constant radial engagement (10–25% of tool diameter) with frequent direction reversals.

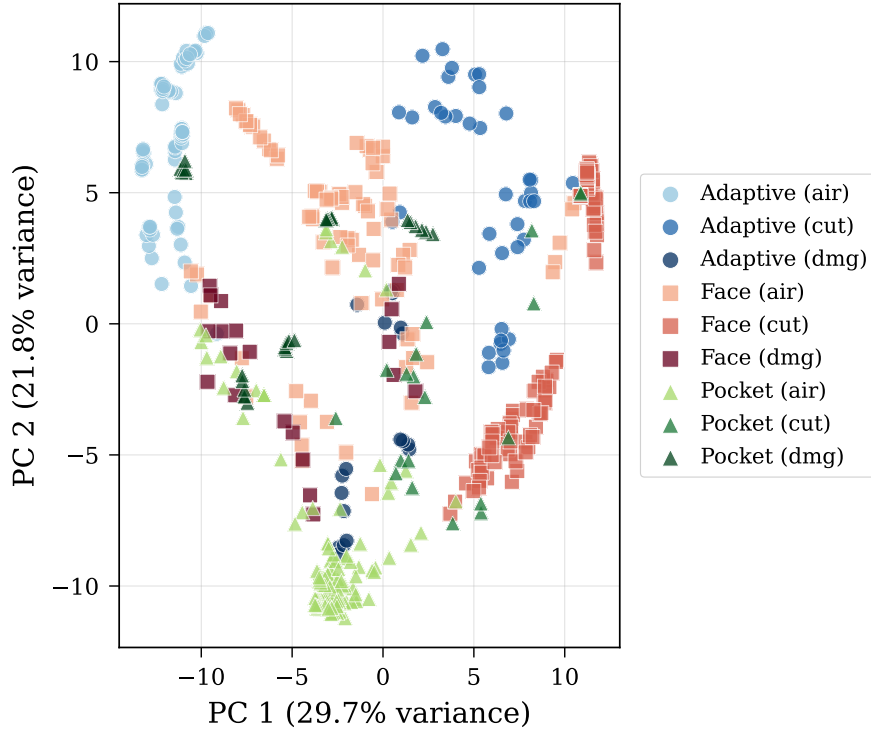
The three conditions are: **air-cutting** (spindle running, no material engagement—machine dynamics only), **active cutting** (0.635 mm depth in UHMW-PE, adding cutting forces and chip formation signatures), and **damaged spindle** (air-cutting with one drive band removed, inducing spindle runout that simulates one specific drive-train degradation mode). We emphasize that this damaged condition is a single simulated fault; real-world spindle health issues include bearing wear, unbalance, and thermal growth, whose sensor signatures may differ from the drive-band removal we study. The “150025” suffix in class labels encodes the 0.025" depth-of-cut parameter from the CAM configuration.

With window size  $W = 256$  timesteps (64 seconds at  $\sim 4$  Hz) and stride 64, the 120 files produce 545 sensor windows. Each window maps to a single G-code line. Across all folds, 34 unique G-code target sequences exist (27 per fold after stratified splitting), with a highly imbalanced distribution: the most common sequence (G1 X-0.1254) covers 20% of windows, while 17 sequences have fewer than 20 windows each.

Figure 5 shows representative sensor traces from the three toolpath strategies. Adaptive clearing produces irregular, high-frequency direction changes; face milling shows periodic step-over transitions; pocket machining exhibits burst-like plunge-clear dynamics. These distinct temporal patterns are what the encoder compresses into the 256-dimensional embeddings that serve as decoder input. Figure 6 confirms that the encoder separates operation classes in embedding space, with clear clustering by toolpath strategy and subclustering by operating condition.



**Figure 5.** Raw sensor traces for three toolpath strategies (accelerometer magnitude, gyroscope magnitude, RMS audio from one sensor board). Adaptive clearing shows irregular direction changes, face milling shows periodic step-overs, and pocket machining shows burst-like plunge-clear dynamics.



**Figure 6.** PCA of encoder embeddings (mean-pooled over 256 timesteps) for all 552 test windows across five folds. PC1–PC2 explain 51.5% of variance. Operations cluster by toolpath strategy (adaptive/face/pocket) with subclusters by condition (air-cut/active/damaged).

### 6.2. Preprocessing and Cross-Validation

Five-fold file-level stratified cross-validation prevents window-level data leakage. Each fold allocates 69–75 files for training, 21–26 for validation, and 21–26 for testing. A sequence coverage repair algorithm swaps files between folds to ensure all G-code sequences in the validation and test sets also appear in training, eliminating impossible predictions.

Sensor features are robust-scaled (per-fold) using training statistics only. G-code targets are tokenized using the hybrid tokenizer [5] with a 712-token vocabulary.

### 6.3. Training Configuration

All experiments use AdamW optimization with cosine annealing and linear warmup. Key fixed hyperparameters (determined through progressive sweeps): learning rate  $3 \times 10^{-4}$ ,  $d_{\text{model}} = 384$ , batch size 32, dropout 0.1, focal  $\gamma = 2.0$ . Gradient clipping at 1.0 prevents instability. All sweeps ran on a workstation with two NVIDIA RTX A6000 GPUs (48 GB each), using Ray Tune for parallel trial scheduling (6–14 concurrent trials). Table 4 summarizes the computational budget.

### 6.4. Sweep Methodology

Rather than a single large sweep, we conducted seven progressive sweeps (V1–V7), each building on insights from the previous:

1. **V1** (1,000 trials): Baseline architecture search. 100 epochs, single fold.
2. **V2** (892 trials): Wider architectures, higher digit weights. 250 epochs.
3. **V3** (320 trials): Cross-validation, longer training. 500 epochs, 5 folds.
4. **V4** (352 trials): +hierarchical conditioning, +memory positional encoding.
5. **V5** (1,568 trials): +grammar constraints, +multi-window context.
6. **V6** (493 trials): Fixed grammar rules, +oversampling, +window dropout.

**Table 4.** Computational Budget ( $2 \times$  NVIDIA RTX A6000). Approximately 2,000 GPU-hours over two weeks, with individual trials requiring 10–45 minutes.

| Experiment               | Runs         | GPU-hrs                        | Min/run  |
|--------------------------|--------------|--------------------------------|----------|
| V1–V2 sweeps             | 1,892        | 390                            | 10–15    |
| V3–V6 sweeps             | 2,733        | 1,370                          | 25–35    |
| V7 sweep                 | 157          | 92                             | 35       |
| 5-fold retrain (5 seeds) | 25           | 13                             | 30       |
| Architecture ablations   | 60           | 30                             | 30       |
| Modality ablations       | 45           | 28                             | 35       |
| Window ablations         | 10           | 5                              | 30       |
| Baselines                | 35           | 5                              | 8        |
| Encoder training         | 50           | 2                              | 2        |
| <b>Total</b>             | <b>5,007</b> | <b><math>\sim 1,935</math></b> | <b>—</b> |

7. **V7 (157 trials):** Re-preprocessed data from raw CSVs; +window position, +file-level MWC, +pointer network, +sensor prior.

The progressive approach locks converged hyperparameters early (*e.g.*,  $d_{\text{model}} = 384$  locked after V2, focal  $\gamma = 2.0$  after V3) and introduces new features in controlled batches, enabling causal attribution of accuracy improvements. With the experimental framework established, the following section presents results addressing each research question.

## 7. Results

This section presents results addressing the four research questions in order: reconstruction accuracy (RQ1, Section 7.1), the relative contribution of architecture versus data handling (RQ2, Section 7.2), sensor modality importance (RQ3, Section 7.3), and minimum viable configuration for deployment (RQ4, Section 7.4). Throughout, we report metrics from the final V7 sweep (157 trials, 5-fold cross-validation) unless otherwise noted. The sweep progression from V1 through V7 and additional feature analyses are detailed in Appendix C.

An important caveat up front: a position-based lookup table achieves 90.4% sequence accuracy on this dataset without any model, outperforming the decoder’s 86.1% (Table 8). The decoder’s contribution is not raw accuracy on the training geometry—a lookup table achieves that more efficiently—but rather a learned sensor-to-G-code mapping that generalizes beyond the training file structure and provides structured predictions suitable for anomaly detection.

### 7.1. Reconstruction Accuracy (RQ1)

Table 5 reports the per-fold results for the best V7 configuration, retrained from scratch on each fold with 5 random seeds per fold (best seed selected).

The decoder achieves **84.5% mean cross-fold sequence accuracy** (single-seed average, 5 seeds per fold). The best-of-5-seeds figure is 86.1% and is reported alongside for comparison, but the single-seed mean is the more honest estimator of deployable performance because it does not include seed-selection bias. Token accuracy is 94.6% and command accuracy is 98.3%—the structural components of G-code (motion commands and parameter addresses) are near-perfectly reconstructed; the remaining errors concentrate in numeric coordinate values, particularly Y-axis step-over positions (Section 8).

Uncertainty on headline numbers.

With only  $n = 5$  folds, Gaussian standard errors are not reliable summaries of uncertainty. A paired fold-level bootstrap (10,000 resamples) gives the following 95% confidence intervals: mean-of-seeds sequence accuracy 84.5%, 95% CI [76.8, 88.8]; best-of-seeds 86.1%, 95% CI [78.5, 90.5]; sensor-only (window position removed) 62.3%, 95% CI [57.1, 67.4]; physics-ceiling gap 22.2 pp, 95% CI

**Table 5.** V7 Per-Fold Test Results (%). Each fold was retrained with 5 random seeds; we report the mean-of-seeds as the primary per-fold value and the best-of-seeds for comparison. Std reflects fold-to-fold variance; within-fold seed std averages 1.4 pp. Fold 1 is consistently the hardest—an encoder-representation limitation not addressed by the decoder.

|             | Seq (mean)  | Seq (best)  | Tok         | Num         | Cmd         | Param       |
|-------------|-------------|-------------|-------------|-------------|-------------|-------------|
| Fold 1      | 69.4        | 71.2        | 87.6        | 80.4        | 96.3        | 92.9        |
| Fold 2      | 88.3        | 89.4        | 96.3        | 93.7        | 97.6        | 98.4        |
| Fold 3      | 87.4        | 88.7        | 95.6        | 93.0        | 100.0       | 98.1        |
| Fold 4      | 88.0        | 89.8        | 96.3        | 93.3        | 97.7        | 98.5        |
| Fold 5      | 89.5        | 91.4        | 97.2        | 93.8        | 100.0       | 98.2        |
| <b>Mean</b> | <b>84.5</b> | <b>86.1</b> | <b>94.6</b> | <b>90.8</b> | <b>98.3</b> | <b>97.2</b> |
| Std         | 8.5         | 8.4         | 4.1         | 5.7         | 1.7         | 2.4         |

[17.9, 27.4]. The wide CIs reflect the limited statistical power of 5-fold CV on a 545-window dataset; the gap CI nonetheless excludes zero with comfortable margin, confirming that the physics-ceiling effect is robust to fold resampling even though individual accuracy numbers are not tightly estimated. All other quantitative claims in this paper should be read under similar caveats: point estimates are reliable to within  $\pm 5$  pp, and we recommend interpreting differences smaller than the widest individual CI (about 6 pp for the best-of-seeds headline) as qualitative rather than quantitative.

Teacher-forced and greedy autoregressive decoding produce identical aggregate sequence accuracy on all five folds, and beam search ( $k \in \{3, 5\}$ ) provides no further improvement. Per-item inspection reveals that individual predictions do differ across decode modes on 7.1% of test windows (median across folds; 18% on the hardest fold), but those differences are symmetric—one decode mode gets right what another gets wrong, in equal counts—so aggregate accuracy is unchanged. This is the signature of representational ambiguity at the sensor level rather than search error in the decoder: any reasonable decoding strategy fails on the same set of physically ambiguous windows, just on slightly different subsets. The scheduled sampling training regime ( $p_{ss} = 0.5$ ) further closes the teacher-forcing gap, so the exposure-bias concern that usually distinguishes teacher-forced evaluation from greedy decoding is empirically absent here.

Table 6 reports per-operation accuracy, computed from the V7 best configuration with teacher-forced decoding across all 552 test windows.

**Table 6.** Reconstruction Accuracy by Operation Class (V7 Best Config, 552 Test Windows)

| Toolpath       | Condition | Windows    | Seq.                     | Tok.         |
|----------------|-----------|------------|--------------------------|--------------|
| Adaptive       | Air-cut   | 40         | 100.0%                   | 100.0%       |
| Adaptive       | Active    | 68         | 91.2%                    | 95.6%        |
| Adaptive       | Damaged   | 20         | 100.0%                   | 100.0%       |
| Face           | Air-cut   | 144        | 100.0%                   | 100.0%       |
| Face           | Active    | 87         | 72.4%                    | 89.6%        |
| Face           | Damaged   | 20         | 90.0%                    | 96.7%        |
| Pocket         | Air-cut   | 19         | 94.7%                    | 99.1%        |
| Pocket         | Active    | 124        | 71.0%                    | 85.9%        |
| Pocket         | Damaged   | 30         | 60.0%                    | 77.6%        |
| <b>Overall</b> |           | <b>552</b> | <b>85.3%<sup>†</sup></b> | <b>93.5%</b> |

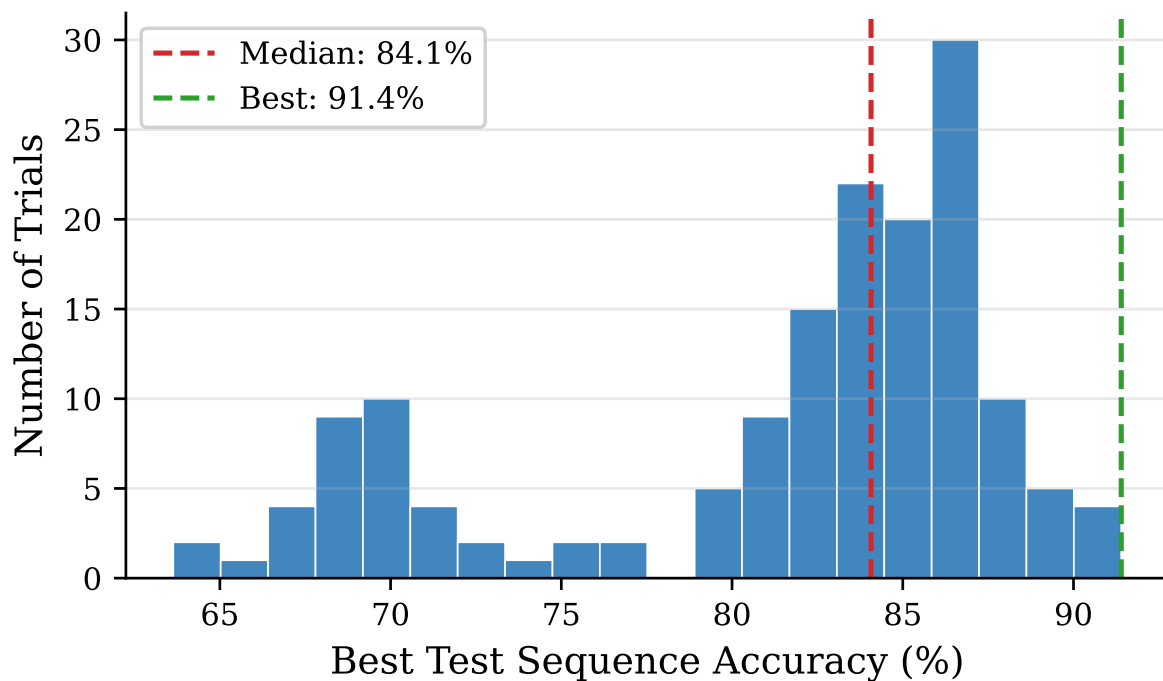
<sup>†</sup>Pooled across all windows; differs from the 86.1% mean-of-folds (Table 5) because fold sizes are unequal and the hardest fold (Fold 1, 71.2%) has the most test windows (132 of 552).

Reconstruction difficulty varies systematically with both toolpath strategy and operating condition. Air-cutting (no material engagement) is near-perfect across all three strategies (99.5% mean sequence accuracy): machine dynamics are highly repeatable when no cutting forces are present.

Active cutting (76.3%) and damaged spindle (80.0%) are 6–24 pp harder because cutting-force variability from chip formation and spindle runout introduce noise that obscures the coordinate-level signal. By strategy, adaptive clearing (95.3%) and face milling (89.6%) outperform pocket machining (71.7%). Pocket air-cutting remains high (94.7%) because without material, the plunge-clear geometry still produces distinctive motion dynamics. Active pocket drops to 71.0% because chip formation forces vary with instantaneous radial engagement during lateral clearing passes, adding stochastic noise to the sensor signal that obscures the coordinate-level information the decoder needs for exact reconstruction.

Fold variance.

V7's data pipeline fixes reduced the cross-fold spread from 26.3 pp (V6) to 17.9 pp and reordered fold difficulty entirely: Fold 5, previously the weakest (52.7% in V6), became the strongest (91.4% in V7) because window position metadata resolved the Y-axis step-over ambiguity that disproportionately affected that fold's test distribution. Fold 1 remains the hardest (71–74% across configurations), suggesting that its file combinations contain toolpaths with inherently ambiguous sensor signatures. The full fold-by-sweep comparison is in Table A1 (Appendix C).



**Figure 7.** Distribution of test sequence accuracy across all 157 V7 trials. The V7 median (84.1%) exceeds V5's best accuracy on four of five individual folds (63.6%, 81.0%, 79.0%, 65.6%), indicating that data pipeline correctness yields more robust performance than architecture search.

Robustness is best captured by the *median*: 84.1% median sequence accuracy versus 33.4% (V5) and 48.4% (V6) (Figure 7). The data pipeline fixes made the decoder robust to hyperparameter choices: nearly every configuration now produces a viable model, whereas in V5 only the top 10% of configurations exceeded 80%.

Statistical significance.

The Wilcoxon signed-rank test yields  $p = 0.0625$  for both V7 vs. V5 and V7 vs. V6 (Table 7). While this narrowly misses the conventional  $\alpha = 0.05$  threshold, it is the *minimum possible p-value* with  $n = 5$  paired samples when all differences are in the same direction—meaning V7 improved every fold for both comparisons. The practical significance is clear: V7 reduced the standard deviation from 10.0 (V5) to 7.2 while simultaneously raising the mean by 10.9 percentage points.

**Table 7.** Best-Per-Fold Sequence Accuracy: Mean  $\pm$  Std (%). Wilcoxon  $p = 0.0625$  is the minimum possible with  $n=5$  paired samples when all differences are positive.

|           | Mean $\pm$ Std                   | Min–Max          | vs. V7 $p$ |
|-----------|----------------------------------|------------------|------------|
| V5        | 75.1 $\pm$ 10.0                  | 63.6–86.4        | 0.0625     |
| V6        | 66.0 $\pm$ 10.4                  | 52.7–79.0        | 0.0625     |
| <b>V7</b> | <b>86.1 <math>\pm</math> 8.4</b> | <b>71.2–91.4</b> | —          |

Wilcoxon signed-rank test (paired by fold). With  $n=5$  paired samples,  $p=0.0625$  is the minimum achievable  $p$ -value when all differences are positive (all 5 folds improved V5→V7 and V6→V7).

## 7.2. Architecture vs. Data Pipeline (RQ2)

A central finding of this work is that data-handling correctness dominates architectural complexity for reconstruction accuracy. We present evidence from baselines, controlled ablations, and the sweep history.

### 7.2.1. Baseline Comparisons

To quantify the decoder’s contribution over simpler approaches, we compare against a progression of baselines (Table 8).

**Table 8.** Baseline Comparison (mean  $\pm$  std across 5 folds). The position-based lookup table (1A) outperforms the decoder on raw accuracy; model-based baselines (1B–1D) perform 33–40 pp worse.

| Method                                       | Seq.                             | Num.                             | Cmd.                             | Tok.                             |
|----------------------------------------------|----------------------------------|----------------------------------|----------------------------------|----------------------------------|
| 1A. Majority lookup*                         | 90.4 $\pm$ 5.6                   | 92.2 $\pm$ 5.0                   | 98.7 $\pm$ 1.1                   | 96.3 $\pm$ 2.4                   |
| 1B. 1-NN retrieval                           | 46.5 $\pm$ 4.1                   | 52.5 $\pm$ 6.2                   | 92.0 $\pm$ 6.1                   | 75.2 $\pm$ 4.3                   |
| 1B. 5-NN retrieval                           | 50.3 $\pm$ 3.6                   | 57.2 $\pm$ 4.7                   | 93.1 $\pm$ 6.1                   | 78.1 $\pm$ 5.1                   |
| 1C. MLP (1-layer)                            | 48.9 $\pm$ 4.5                   | 55.3 $\pm$ 6.0                   | 92.9 $\pm$ 6.4                   | 77.8 $\pm$ 3.7                   |
| 1C. MLP (2-layer)                            | 47.4 $\pm$ 3.3                   | 53.2 $\pm$ 5.1                   | 94.0 $\pm$ 4.5                   | 77.2 $\pm$ 4.4                   |
| 1D. Random Forest                            | 53.5 $\pm$ 3.5                   | 60.9 $\pm$ 3.4                   | 93.7 $\pm$ 6.7                   | 80.2 $\pm$ 4.0                   |
| 1D. XGBoost                                  | 53.0 $\pm$ 5.3                   | 60.0 $\pm$ 5.5                   | 94.1 $\pm$ 5.9                   | 79.5 $\pm$ 4.5                   |
| 1E. 34-class MLP (sensor-only) <sup>‡</sup>  | 49.6 $\pm$ 4.9                   | —                                | —                                | —                                |
| 1E. 34-class MLP (+window pos.) <sup>‡</sup> | 77.5 $\pm$ 8.1                   | —                                | —                                | —                                |
| <b>V7 decoder (mean-of-seeds)</b>            | <b>84.5 <math>\pm</math> 8.5</b> | <b>90.8 <math>\pm</math> 5.7</b> | <b>98.3 <math>\pm</math> 1.7</b> | <b>94.6 <math>\pm</math> 4.1</b> |
| V7 decoder (best-of-seeds)                   | 86.1 $\pm$ 8.4                   | 91.3 $\pm$ 5.4                   | 98.1 $\pm$ 1.9                   | 94.9 $\pm$ 4.2                   |

\*Position→majority G-code lookup table, no model.

<sup>‡</sup>1E: MLP(→256→128→34) on mean-pooled encoder memory, 34-way sequence classification. The two rows differ only by whether the window position features (index, total) are appended to the input. This confirms that a simple classifier can extract most of the position signal; the decoder’s 7 pp improvement over 1E(+pos) reflects the value of autoregressive grammar decomposition on top of the same features. All baselines except 1A use frozen encoder embeddings.

A counterintuitive result emerges from the baselines: the position-based majority lookup table (1A) achieves 90.4% mean sequence accuracy—*higher than either the decoder or any other baseline* on this fixed-geometry dataset. Subset analysis confirms the lookup table’s advantage is not limited to unambiguous cases: it achieves 92.6% on windows with a unique G-code at that position and 82.9% on ambiguous windows (multiple G-codes sharing the same position), compared to the decoder’s 88.9% and 77.7% respectively. On this specific dataset, the window-position-to-G-code mapping is so close to deterministic that a zero-parameter lookup exploits it more efficiently than any learned model.

We do not treat this as an embarrassment to report but as the central scientific finding of Section 7.2. The lookup table’s 90.4% is an upper bound on what any method restricted to the fixed geometry of this dataset can achieve: it is exactly what you get when you let the file structure speak directly,

without any sensor evidence at all. The decoder’s 84.5% is what you get when you do the same thing with a 22-million-parameter learned map instead—worse, because cross-attention is a less efficient way to exploit a deterministic index. The question the rest of this paper asks is not “does the decoder beat the lookup on the training geometry?” (it does not, and should not be expected to) but rather: *what does the decoder preserve that the lookup table does not?*

Three answers follow from the baselines and the held-out-toolpath experiments (Section 7.5). First, the decoder does not require the (source\_file, window\_index) mapping that the lookup table depends on; it can reconstruct from sensor content alone, reaching 62.3% sequence accuracy on the pure-sensor-only configuration (Table 9, sensor-only row), whereas the lookup table has no sensor-only mode. Second, when the training distribution is held out (Section 7.5), the lookup table collapses on every metric to essentially zero, while the decoder retains command accuracy up to 89% and parameter-type accuracy up to 82% on unseen toolpath classes—substantially more of the G-code grammar transfers across toolpath families than the lookup can capture. Third, the decoder produces token-level structured predictions (type, command, parameter letter, digit positions) that are the natural interface for downstream anomaly detection; the lookup produces only a whole-sequence label.

The simpler model-based baselines (1B–1D) all perform 33–40 pp worse than both the lookup table and the decoder. Row 1E adds a 34-class MLP trained directly on pooled encoder memory: without window-position features it reaches 49.6%, consistent with 1B–1D; with window-position features appended it jumps to 77.5%. The gap from 77.5% (MLP + position) to 84.5% (decoder) quantifies what the autoregressive grammar decomposition adds on top of the same input features—a genuine but modest improvement, consistent with the physics-ceiling argument.

### 7.2.2. Ablation Studies

Each V7 component’s contribution is isolated by toggling one feature at a time from the best configuration, reporting mean  $\pm$  std across all 5 folds.

**Table 9.** Ablation Study (mean  $\pm$  std across 5 folds, best V7 config)

| Ablation                                    | Seq. (%)       | $\Delta$ | Cat.           |
|---------------------------------------------|----------------|----------|----------------|
| Sensor+position (full V7 base) <sup>†</sup> | 86.0 $\pm$ 7.2 | —        | —              |
| <i>Sensor-only</i> (no window pos.)         | 62.3 $\pm$ 5.9 | −23.8    | <i>Physics</i> |
| <i>Data pipeline ablations</i>              |                |          |                |
| 2A1. No window position                     | 62.3 $\pm$ 5.9 | −23.8    | Data           |
| 2A2. No file-level MWC                      | 83.2 $\pm$ 7.9 | −2.8     | Data           |
| 2A3. No sched. sampling                     | 84.4 $\pm$ 6.8 | −1.6     | Train          |
| 2A4. All data features OFF                  | 61.6 $\pm$ 8.3 | −24.5    | Data           |
| <i>Architecture ablations</i>               |                |          |                |
| 2B1. No mem. pos. enc.                      | 84.7 $\pm$ 7.4 | −1.3     | Arch           |
| 2B2. No sensor prior                        | 83.9 $\pm$ 7.5 | −2.1     | Arch           |
| 2B3. No focal loss                          | 84.8 $\pm$ 6.9 | −1.2     | Arch           |
| 2B4. No grammar                             | 85.1 $\pm$ 7.0 | −0.9     | Arch           |
| 2B5. No label smoothing                     | 85.2 $\pm$ 6.6 | −0.8     | Arch           |
| <i>Capacity ablations</i>                   |                |          |                |
| 2C1. Shallow ( $n_L=2$ )                    | 84.4 $\pm$ 7.6 | −1.6     | Cap            |
| 2C2. Narrow ( $d=192$ )                     | 84.7 $\pm$ 6.5 | −1.3     | Cap            |
| 2C3. Fewer heads ( $h=4$ )                  | 85.0 $\pm$ 7.1 | −1.0     | Cap            |

The *Sensor-only* row reproduces the 2A1 ablation (window position removed) and serves as the anchor for the physics-ceiling analysis: it quantifies what the decoder can recover from sensor dynamics alone, without any within-file metadata. <sup>†</sup> Ablation base is separate training runs; differs from Table 5 by 0.1 pp seed variance.

A stark asymmetry emerges (Table 9). The window-position-only ablation drops accuracy by 23.8 pp—within rounding of the 24.5 pp drop when all three data features are removed together, so window position accounts for essentially the entire “data-pipeline” contribution. The incremental lift from file-level multi-window context and scheduled sampling beyond window position is only 0.7 pp combined. No architectural feature—memory positional encoding, sensor prior, focal loss, grammar constraints, label smoothing—exceeds a 2.1 pp individual contribution, and all are within one standard deviation of the base configuration. Capacity ablations behave similarly: a 3.0M-parameter decoder ( $n_L=2$ ,  $d=192$ ,  $h=4$ ) reaches 84.4%, just 1.6 pp below the full 22.1M-parameter architecture.

We interpret these numbers as evidence of a physical ceiling rather than an architectural one. The sensor-only row (62.3%) is what a decoder with full access to the sensor embeddings and every other architectural feature achieves when window-position metadata is withheld—that is, the best possible single-window sensor-to-G-code recovery under this encoder. The remaining gap to 86.0% is closed by a single ordinal feature (window index within the source file), a feature that a lookup table can exploit even more directly (hence the 90.4% lookup table in Section 7.2). No amount of architectural sophistication closes this gap, because the missing information is not in the sensors. This is consistent with the information-theoretic analysis in Section 7.6 showing that window position provides 57.8% normalized information gain for Y-axis disambiguation.

The train-val gap is also revealing. Across all 25 multi-seed V7 runs the mean final train-val token-accuracy gap is 2.5% (median 1.4%, max 7.6%); Fold 1 is highest at 6.2%, consistent with its being the hardest to generalize on. A 22.1M-parameter decoder trained on 545 samples would ordinarily be expected to show a substantial overfitting gap; the near-zero gap we observe instead is further evidence that the binding constraint is the encoder’s representational capacity, not the decoder’s parameter count. Full training curves for all 25 runs are in Figure A1.

A two-way repeated-measures ANOVA (13 conditions  $\times$  5 folds) confirms that ablation condition is a highly significant factor ( $F(12, 48) = 28.89$ ,  $p < 0.001$ ,  $\eta_p^2 = 0.878$ ), as is fold ( $F(4, 48) = 53.91$ ,  $p < 0.001$ ,  $\eta_p^2 = 0.818$ ). With only one observation per condition-fold cell, a formal interaction  $F$  cannot be separated from residual error; as a principled proxy we ran Tukey’s 1-df test for nonadditivity, which is significant ( $F(1, 47) = 7.91$ ,  $p = 0.007$ ), indicating that the ranking of ablation effects varies systematically across folds. Bonferroni-corrected post-hoc paired  $t$ -tests (12 comparisons) confirm that only the window-position ablation reaches significance after correction ( $p_{\text{corr}} = 0.015$ , Cohen’s  $d = -3.64$ ); the remaining 11 ablations produce Cohen’s  $d$  values between  $-0.57$  and  $-2.30$  (medium to large effects) but are underpowered at  $n = 5$  folds.

These findings push back on the common assumption that architectural novelty is what drives accuracy in sensor-to-sequence tasks. On this dataset, one ordinal metadata feature (window position) contributes more than the sum of all learned architectural components. The practical implication is that practitioners should invest in data preparation and metadata correctness before architecture search—and that architecture ablations on small, geometrically constrained datasets can give a misleading picture of which design choices matter.

Grammar constraints, analyzed in detail during the V5–V6 sweeps (Appendix C.5), show a nuanced pattern: after fixing rule correctness to allow modal persistence (BOS→PARAMETER transitions), grammar helps the best-tuned models achieve higher peaks (+0.9 pp mean in V7) but can propagate commitment errors in undertrained models.

### 7.2.3. Sweep Progression and Data Pipeline Dominance

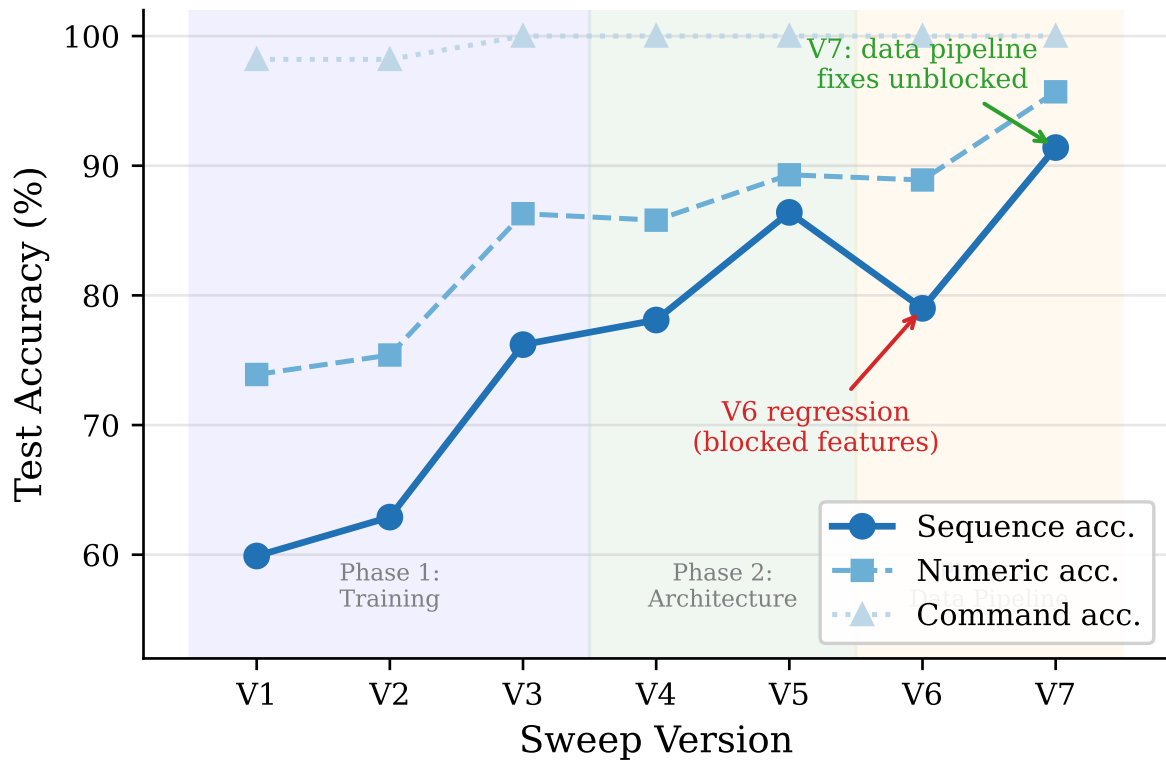
Table 10 summarizes the best test-set results across all seven sweeps, spanning 4,782 total trials.

The progression (Figure 8) reveals three distinct phases: training sufficiency (V1→V3, +16.3 pp), temporal context (V3→V5, +10.2 pp via MWC and memory positional encoding), and data pipeline correctness (V5→V7, +5.0 pp best, +36 pp median). The detailed per-sweep narrative is in Appendix C. V6 regressed to 79.0% because three high-impact data features were structurally blocked by missing metadata in preprocessed files: window position fell back to zero-vectors, MWC used random

**Table 10.** Decoder Sweep Progression (Best Test-Set Results). Each sweep builds on the previous, locking converged parameters and introducing new features.

| Sweep | Trials | Seq.              | Num. | Cmd. | Key Addition             |
|-------|--------|-------------------|------|------|--------------------------|
| V1    | 1,000  | 59.9              | 73.9 | 98.2 | Baseline                 |
| V2    | 892    | 62.9              | 75.4 | 98.2 | Wider $d_{\text{model}}$ |
| V3    | 320    | 76.2              | 86.3 | 100  | 5-fold CV, 500 ep.       |
| V4    | 352    | 78.1              | 85.8 | 100  | +hier., +mem. pos. enc.  |
| V5    | 1,568  | 86.4              | 89.3 | 100  | +MWC, +mem. pos. enc.    |
| V6    | 493    | 79.0 <sup>†</sup> | 88.9 | 100  | Fixed grammar rules      |
| V7    | 157    | 91.4              | 95.7 | 100  | +win. pos., file MWC     |

<sup>†</sup>V6 regressed: three data pipeline features were structurally blocked (see text).



**Figure 8.** Sweep progression across V1–V7. Three phases are visible: training sufficiency (V1–V3), architectural improvements via temporal context (V3–V5), and data pipeline correctness (V5–V7). V6 regressed due to blocked data features; V7 recovered and set a new best.

(post-shuffle) neighbors instead of file-local windows, and non-stratified splits left 44 G-code sequences absent from certain folds' training sets. V7 re-processed data from raw CSVs, unblocking all three. The result is the largest improvement in *robustness*: the median sequence accuracy rose from 33% (V5) and 48% (V6) to 84% (V7), meaning that nearly every hyperparameter configuration now produces a viable model. This episode demonstrates that data pipeline correctness can dominate architectural improvements.

### 7.3. Sensor Modality Importance (RQ3)

Having established the baseline accuracy and the dominance of data pipeline features, we turn to the question of which sensor modalities are essential for reconstruction.

#### 7.3.1. Modality Ablation

To determine which sensor modalities are essential for reconstruction, we retrain the full pipeline (encoder + decoder) nine times, each time excluding one modality while keeping all others. Table 11 reports the results.

**Table 11.** Sensor Modality Ablation: Effect of Removing One Modality

| Removed                    | <i>F</i> | Seq. (%)        | $\Delta$ | Class. Rank |
|----------------------------|----------|-----------------|----------|-------------|
| None (base, f110)          | 110      | $86.1 \pm 8.4$  | —        | —           |
| Gyroscope <sup>†</sup>     | 92       | $58.4 \pm 34.5$ | −27.7    | #6          |
| Color (RGBA)               | 86       | $58.9 \pm 8.7$  | −27.2    | #5          |
| Accelerometer <sup>†</sup> | 92       | $63.1 \pm 35.8$ | −23.0    | #7          |
| Magnetometer               | 92       | $68.7 \pm 13.0$ | −17.4    | #4          |
| Electrical                 | 102      | $70.9 \pm 8.5$  | −15.2    | #8          |
| Pressure                   | 104      | $72.1 \pm 9.6$  | −14.0    | #1          |
| Temperature                | 104      | $73.3 \pm 7.9$  | −12.8    | #9          |
| Proximity                  | 104      | $73.5 \pm 5.8$  | −12.7    | #3          |
| RMS Audio                  | 104      | $79.6 \pm 12.6$ | −6.5     | #2          |

*F* = remaining features. Class. Rank = ranking by ANOVA

*F*-statistic for 9-class operation classification [1].

<sup>†</sup>High std from Fold 5 failure (0%); median: gyroscope 66.4%, accelerometer 76.1%.

Reconstruction and classification rank sensor modalities differently (Table 11; Spearman  $\rho = -0.35$ ,  $p = 0.36$ ,  $n = 9$ —a weak negative correlation that does not reach significance but indicates the two rankings diverge in direction). Three findings stand out:

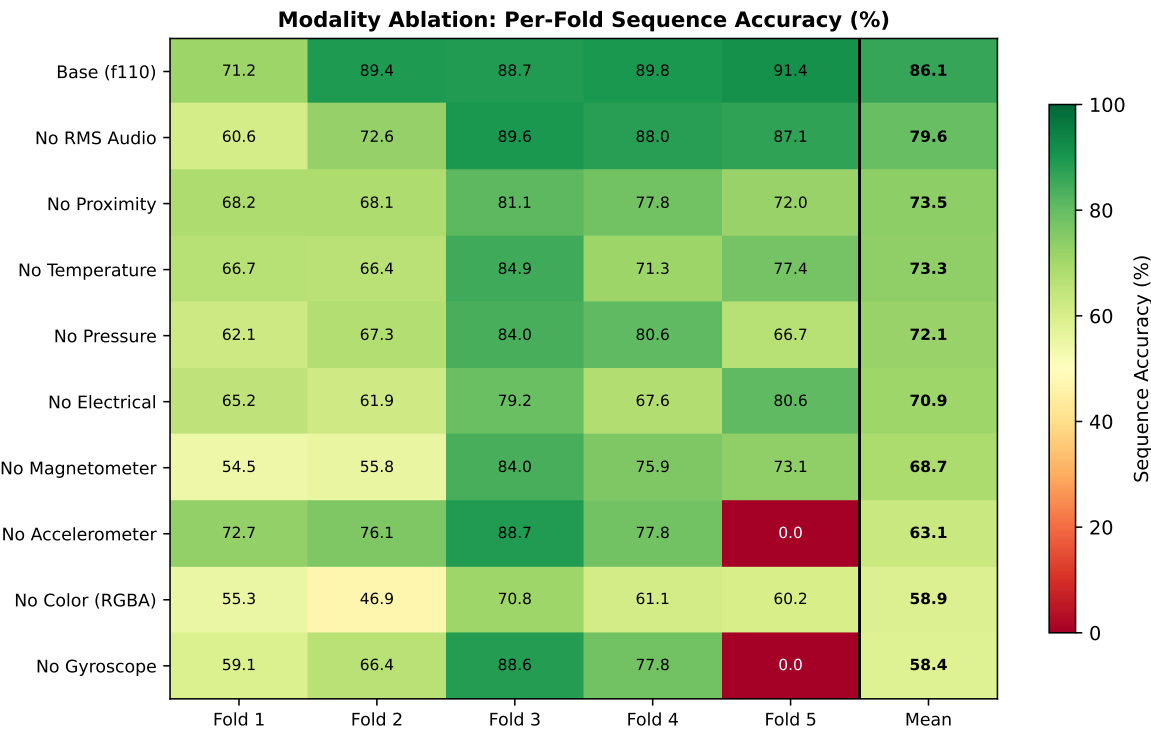
- 1. Inertial sensors dominate reconstruction.** Gyroscope (−27.7 pp) and accelerometer (−23.0 pp) together account for over 50 pp of reconstruction accuracy (Figures 9 and A2). The high standard deviations ( $\pm 34$ – $36\%$ ) reflect complete failure on Fold 5 (0% sequence accuracy) when either is removed, indicating that inertial signals are essential for the window-position-to-G-code mapping on certain file configurations.
- 2. Color is unexpectedly critical (−27.2 pp).** The APDS-9960 RGBA sensor, ranked #5 for classification, is the second most important modality for reconstruction. The mechanism by which removing four low-bandwidth color channels drops reconstruction accuracy by 27 pp is not obvious, and an explicit correlation test (detailed below) ruled out the most natural hypothesis.
- 3. Audio is the least important for reconstruction (−6.5 pp)** despite being the second-strongest classifier feature by ANOVA. Audio's high classification *F*-statistic reflects its ability to distinguish operation *types* (cutting vs. air-cutting), but within an operation type, acoustic signatures carry little coordinate-level information.

The reversal between classification and reconstruction importance has direct implications for sensor selection in deployment. A rig optimized for operation monitoring (classification) would

prioritize pressure and audio; a rig optimized for G-code verification (reconstruction) would prioritize gyroscope, color, and accelerometer. A minimal three-modality reconstruction rig (gyroscope + color + accelerometer, 60 channels) would retain the three largest contributors, though the interaction effects of simultaneous removal remain untested.

The color-hypothesis test.

One natural explanation for the color channel’s reconstruction importance is that ambient light at the sensor boards varies with the table’s Y-coordinate—e.g., because lateral motion changes proximity to the enclosure walls or to fixed illumination sources—providing a direct spatial signal that the decoder exploits for Y-axis disambiguation. To test this, we computed Pearson and Spearman correlations between each of the 24 raw color channels (4 color components × 6 sensor boards) and the machine-reported Y-coordinate (posy), both pooled across all 120 aligned files and within-file to remove between-file setup drift. Pooled correlations are all in the range  $|r| < 0.12$ . The highest within-file mean  $|r|$  is 0.122 (frame\_l3 board, Red channel); the lowest is 0.052 (spindle2, Blue). By the conventional interpretation of correlation strength, none of the color channels meaningfully tracks the Y-coordinate. We therefore reject the ambient-light-gradient hypothesis. The mechanism by which the color sensor contributes 27 pp to reconstruction accuracy remains unexplained. Candidate hypotheses include (a) correlation with operation-class-specific temporal patterns that the encoder’s cross-modal attention amplifies, (b) electromagnetic or thermal cross-talk between the APDS-9960 and the IMU on the same board, and (c) some artifact of the robust scaler’s interaction with the low-variance color signal. We leave identification of the true mechanism to future work.



**Figure 9.** Per-fold sequence accuracy for each modality removal. Gyroscope and accelerometer removal causes catastrophic failure on Fold 5 (0%), while other folds remain viable. Color removal uniformly degrades all folds. Audio removal has the smallest effect across all folds.

### 7.3.2. Encoder Configuration: Confound-Free Reconstruction

The encoder configuration ablation (Table 12) addresses a critical validity concern: the encoder paper [1] identified proximity and pressure channels as barometric confounds (ANOVA  $F > 2000$ ), raising the question of whether reconstruction accuracy is inflated by these non-machining signals.

**Table 12.** Encoder Feature Configuration: Confound-Free Reconstruction. The f98 encoder excludes proximity and pressure channels identified as barometric confounds in [1].

| Config                  | $F$ | Enc. Class. | Seq. Acc.      | $\Delta$ |
|-------------------------|-----|-------------|----------------|----------|
| f110 (full)             | 110 | 92.9%       | $86.0 \pm 7.2$ | —        |
| f98 (no prox/pres)      | 98  | 92.4%       | $81.1 \pm 9.5$ | −4.9     |
| f98 (optimized encoder) | 98  | 94.8%       | $82.5 \pm 9.7$ | −3.5     |

$F$  = number of sensor features. Enc. Class. = encoder 9-class classification accuracy. f98 excludes proximity and pressure channels identified as barometric confounds in [1].

The confound-free f98 encoder (no proximity or pressure) achieves 81.1% mean sequence accuracy—a 4.9 pp drop from the f110 ablation base of 86.0%. Optimizing the f98 encoder’s hyperparameters (yielding 94.8% classification accuracy, *higher* than f110’s 92.9%) recovers only 1.4 pp to 82.5%. This reveals two important findings:

1. **Confound-free reconstruction is viable.** 82.5% sequence accuracy without proximity/pressure channels confirms that the core sensor modalities (accelerometer, gyroscope, temperature, audio, electrical) carry sufficient information for G-code reconstruction.
2. **Better classification  $\neq$  better reconstruction.** The f98 optimized encoder achieves 1.9 pp *higher* classification accuracy than f110 but 3.5 pp *lower* reconstruction accuracy. The proximity/pressure channels carry real signal for coordinate-level discrimination that is not captured by the 9-class classification metric.

### 7.4. Minimum Viable Configuration (RQ4)

For deployment in real manufacturing environments, the full 110-feature, 64-second, 22.1M-parameter configuration may be impractical. This subsection examines how much can be reduced while maintaining viable reconstruction.

#### 7.4.1. Observation Window Size

Table 13 reports reconstruction accuracy as a function of the observation window duration.

**Table 13.** Window Size Ablation. Halving the window from 64 s to 32 s costs 9.2 pp but doubles temporal resolution.

| Window  | Duration | Windows | Seq. (%)       | $\Delta$ |
|---------|----------|---------|----------------|----------|
| $W=256$ | 64 s     | 545     | $86.1 \pm 8.4$ | —        |
| $W=128$ | 32 s     | 861     | $76.9 \pm 5.4$ | −9.2     |
| $W=64$  | 16 s     | 1,964   | $70.6 \pm 5.0$ | −15.5    |

Halving the window from 256 to 128 timesteps (64 s to 32 s) reduces sequence accuracy by 9.2 pp but increases temporal resolution  $1.6\times$  (861 vs. 545 evaluation points per dataset). Quartering to 64 timesteps (16 s) costs 15.5 pp but provides  $3.6\times$  more evaluation points. The 76.9% accuracy at 32 s may be sufficient for structural-level monitoring (command detection remains above 95%), while the full 64 s window is needed for numeric-level coordinate reconstruction.

The accuracy drop is consistent with the encoder bottleneck hypothesis: shorter windows contain less discriminative information for the frozen encoder to compress, reducing the quality of the memory

representation that the decoder attends to. Notably, the fold variance decreases with shorter windows ( $\pm 8.4$  at  $W=256$  vs.  $\pm 5.0$  at  $W=64$ ), suggesting that the hardest fold's difficulty is driven by long-range temporal patterns that shorter windows cannot capture.

#### 7.4.2. Decoder Capacity

Capacity is not the bottleneck. The ablations in Table 9 (conditions 2C1–2C3) confirm that the decoder is overparameterized for this task once data handling is correct. A 2-layer decoder with  $d_{\text{model}} = 192$  and 4 attention heads achieves 84.4%, only 1.6 pp below the full 8-layer,  $d=384$ , 12-head architecture. This minimal decoder has approximately 3.0M parameters—a  $7.4\times$  reduction from the full 22.1M-parameter architecture—making it feasible for edge deployment on embedded hardware alongside the CNC controller.

#### 7.4.3. Deployment Implications

Combining the results above, a minimum viable reconstruction system could use: (1) the confound-free f98 encoder (82.5%), (2) a 32-second observation window (76.9% at full encoder, trading accuracy for  $1.6\times$  temporal resolution), and (3) a 3.0M-parameter decoder (84.4% at full window). These reductions are not independently validated in combination, so interaction effects may further degrade accuracy. However, the structural reconstruction (command + parameter accuracy  $>95\%$ ) remains viable across all reduced configurations, suggesting that a lightweight deployment could reliably detect command-level anomalies (wrong operation type, unexpected parameter) even if coordinate-level precision is reduced.

Figure 10 shows the training dynamics of the best V7 trial. Token-level accuracy converges rapidly ( $\sim 200$  epochs) because structural tokens (commands, parameters) are predicted with high confidence early. Sequence-level accuracy lags because it requires *all* tokens—including the numerically ambiguous ones—to be correct simultaneously. The digit weight  $w_d$ , which controls the relative importance of numeric versus structural loss, shows no significant effect in V7: all three tested values ( $w_d \in \{15, 20, 30\}$ ) achieve the same best accuracy (91.4%) with similar means (81.2%, 81.3%, 82.1% respectively), suggesting that the preprocessing corrections reduced the difficulty of numeric prediction to the point where the loss weighting no longer matters.

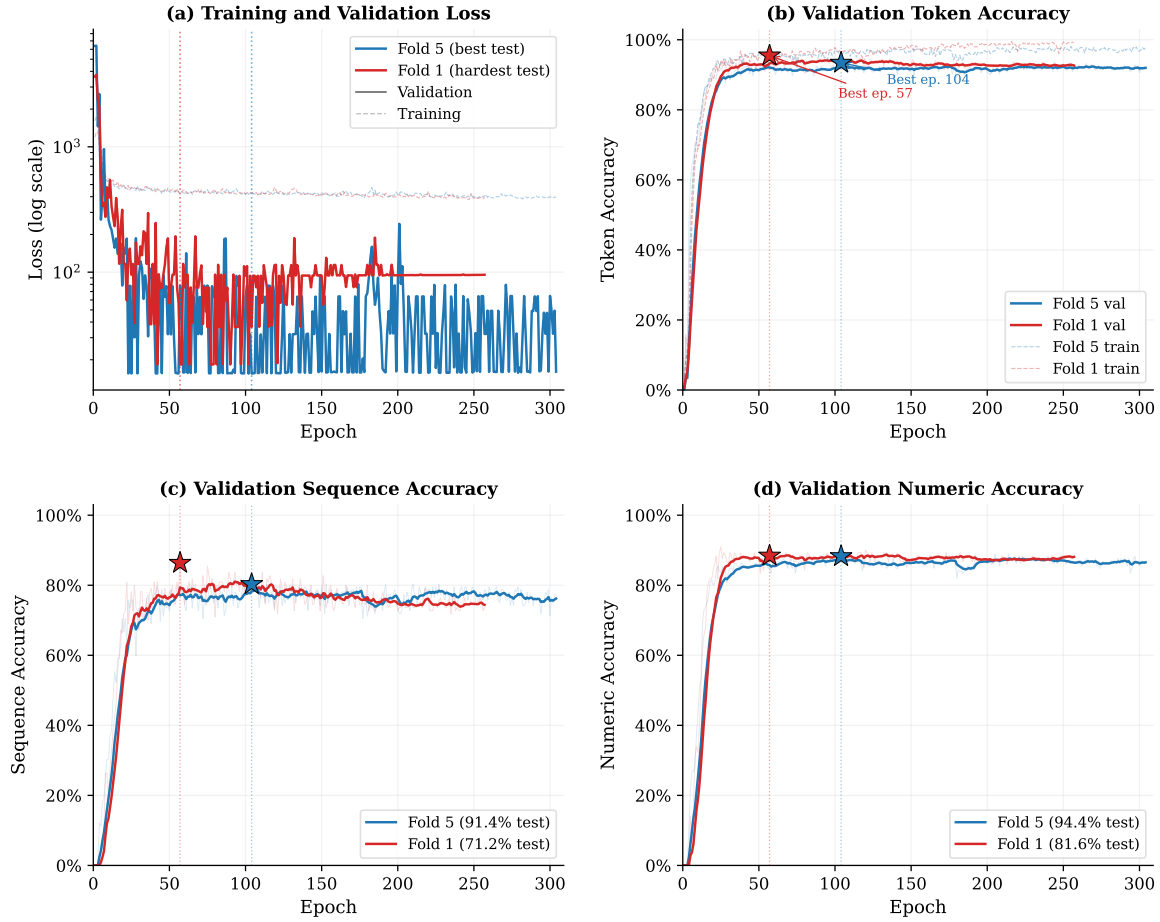
#### 7.5. Generalization to Unseen Toolpath Classes

The cross-validation in Section 7.1 is *in-distribution*: every fold's training set contains files from all three toolpath classes (adaptive, face, pocket), because the stratified round-robin split preserves class balance within each fold. Under those conditions a position-based lookup table reaches 90.4% (Table 8) without any model, because (source\_file, window\_index) is nearly a unique key for the G-code sequence on this fixed-geometry dataset. This leaves open the question of whether the decoder learns anything that a lookup table cannot—and, more scientifically, whether sensor-to-G-code reconstruction transfers at all when the target toolpath class has been withheld from training.

To test this, we construct three leave-one-toolpath-out splits. For each held-out toolpath class  $c \in \{\text{adaptive, face, pocket}\}$  we train the full V7 decoder (reusing the V7 frozen encoder checkpoint) on all files from the two non-heldout toolpath classes, with a stratified 80/20 train/val split, and evaluate on every window from the held-out class. Split sizes are: heldout-adaptive 333/84/128; heldout-face 240/61/244; heldout-pocket 297/75/173 train/val/test windows. The encoder reuses the fold-1 checkpoint; this is a conservative bias because the encoder was originally trained in the 5-fold CV setting and saw some windows from every toolpath—any transfer failure under this setup is a real decoder-level limit, not an encoder-level one.

For the lookup-table baseline on the same splits, the majority map from (window\_index, total\_windows) is built on the non-heldout train $\cup$ val pool and applied to the heldout test pool.

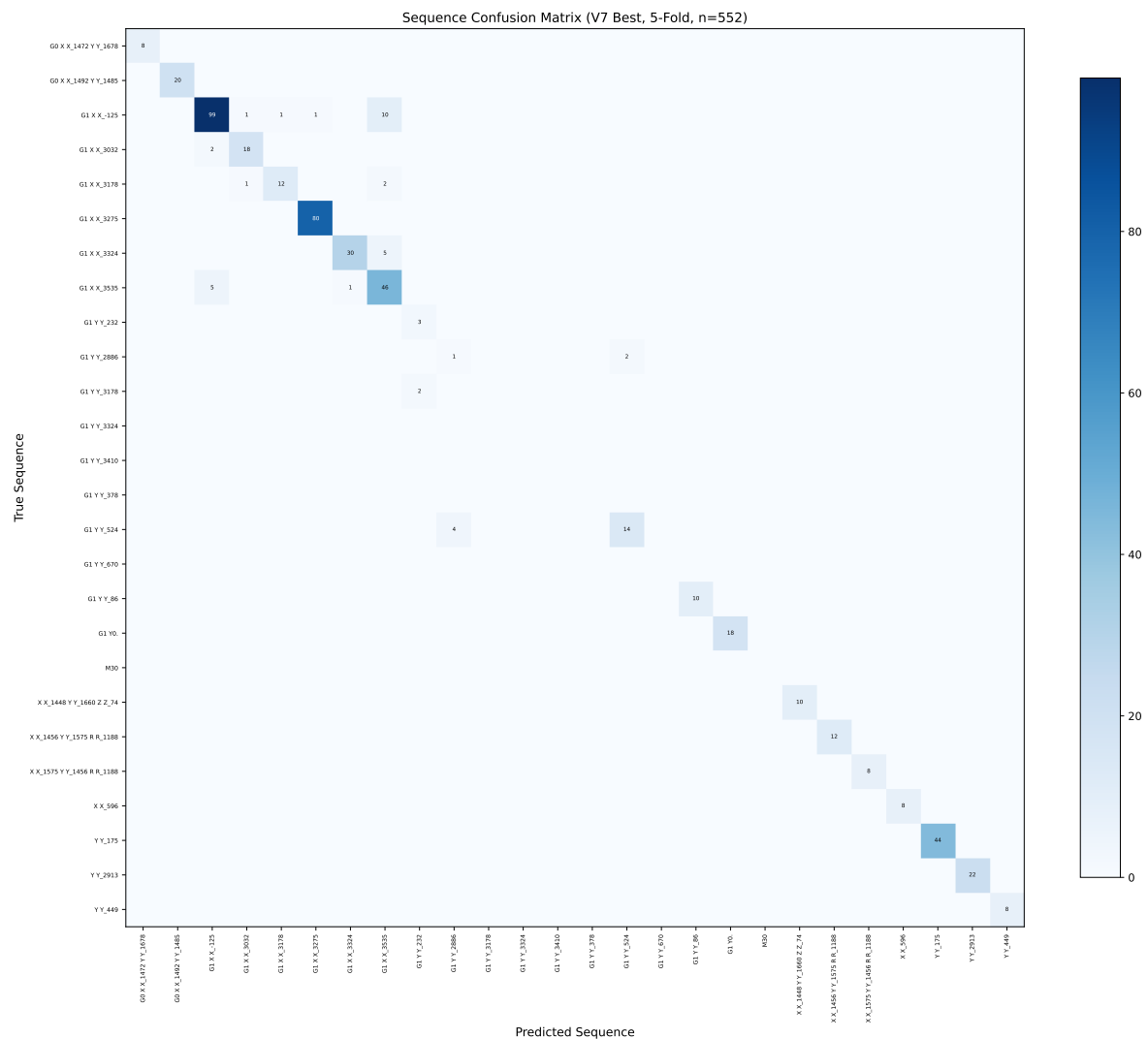
Three findings (Table 14) deserve attention.



**Figure 10.** Validation accuracy over training for the best V7 trial (Fold 5, test sequence accuracy 91.4%). Token and numeric accuracy plateau after ~200 epochs, while sequence accuracy continues to improve slowly through 500 epochs, consistent with the finding that structural prediction converges early while numeric prediction requires longer training.

**Table 14.** Held-Out-Toolpath Generalization (%). Each row is a single-seed run; val reflects performance on the non-heldout distribution, test reflects the held-out toolpath. Both methods produce 0% exact-sequence accuracy on test because held-out toolpaths use numeric coordinate values the model has never seen. The decoder preserves command and parameter-type accuracy at levels substantially above the lookup table across all three splits.

| Heldout  | Method  | Split | Seq  | Tok  | Cmd   | Param | Num  |
|----------|---------|-------|------|------|-------|-------|------|
| adaptive | Decoder | val   | 81.0 | 94.7 | 100.0 | 96.4  | 88.0 |
|          | Decoder | test  | 0.0  | 16.4 | 0.0   | 55.1  | 0.0  |
|          | Lookup  | test  | 0.0  | 0.0  | 0.0   | 0.0   | 0.0  |
| face     | Decoder | val   | 86.9 | 95.6 | 100.0 | 94.6  | 91.9 |
|          | Decoder | test  | 0.0  | 49.3 | 78.3  | 82.6  | 0.0  |
|          | Lookup  | test  | 0.0  | 2.5  | 4.9   | 2.5   | 0.0  |
| pocket   | Decoder | val   | 92.0 | 97.7 | 100.0 | 98.3  | 95.4 |
|          | Decoder | test  | 0.0  | 50.1 | 89.5  | 59.5  | 0.0  |
|          | Lookup  | test  | 0.0  | 36.4 | 80.2  | 33.5  | 0.0  |



**Figure 11.** Sequence-level confusion matrix across all 552 test windows (V7 best config, 5 folds, teacher-forced decoding). The dominant off-diagonal pattern is Y-axis step-over confusion: sequences differing only in  $Y=0.175 \leftrightarrow Y=2.913 \leftrightarrow Y=0.449$ . Face milling and adaptive clearing under air-cutting achieve 100% sequence accuracy. Cross-class confusion is rare; most errors are within-class numeric substitutions in the active-cutting and damaged-spindle conditions.

**Both methods collapse on sequence accuracy.** Neither the decoder nor the lookup table recovers a single exact G-code sequence on any held-out split. This is not a bug. It is the predicted consequence of the physics-ceiling analysis in Sections 7.2 and 7.6: the numeric coordinate values used by the held-out toolpath class are not present in the training vocabulary at all (the 712-token vocabulary is geometry-specific, and each toolpath class generates a disjoint subset of numeric tokens). Exact-sequence reconstruction therefore requires that at least one instance of each target sequence have been seen during training. Neither a lookup table nor the current decoder can violate this constraint.

**Numeric accuracy collapses to zero for the same reason.** The numeric coordinate tokens for the held-out toolpath are never seen during training and thus cannot be predicted by any method that treats coordinates as a closed vocabulary. The digit-by-digit open-vocabulary framework of [5] is a natural remedy and is identified as future work (Section 9).

**Command and parameter-type accuracy, however, transfer substantially—and the decoder transfers them much better than the lookup table.** On heldout-face, the decoder reaches 78.3% command accuracy and 82.6% parameter-type accuracy; the lookup table reaches 4.9% and 2.5%. On heldout-pocket, the decoder reaches 89.5%/59.5%; the lookup reaches 80.2%/33.5%. Even on heldout-adaptive—the hardest split, because adaptive clearing uses distinctive trochoidal arc commands (G2/G3) that face and pocket do not—the decoder preserves 55.1% parameter-type accuracy, against 0.0% for the lookup. Averaged across the three splits, the decoder’s parameter-type accuracy advantage over the lookup table is 53.1 pp, and its command accuracy advantage is 27.3 pp. These gaps measure what the autoregressive grammar decomposition learns that a positional lookup cannot: the toolpath-invariant structure of G-code (motion-command dynamics, parameter-letter regularities) rather than just the toolpath-specific mapping from position to sequence.

**On heldout-adaptive the decoder collapses to 0% command accuracy.** Face and pocket toolpaths primarily use G0 and G1 motion commands; adaptive clearing additionally uses G2/G3 arcs for trochoidal retract moves. A decoder that has never seen G2/G3 during training cannot emit them, and the grammar constraints (which forbid BOS→G2/G3 when those tokens are absent from the training distribution) lock the decoder to a small set of tokens it has learned. This is a clean falsification of any claim that the decoder would generalize to genuinely novel command tokens on this dataset, and it sharpens the scope of what held-out-toolpath transfer actually means: *structural* transfer (parameter letters, modal-persistence grammar) is real, but transfer to genuinely unseen motion-command tokens is not.

The overall message of the held-out-toolpath experiment is narrower and more honest than “the decoder generalizes.” On a fixed-geometry benchmark where positional metadata is nearly deterministic, neither a decoder nor a lookup can produce novel exact G-code sequences. What differs between them is how much of G-code’s universal grammar—motion commands and parameter letters that mean the same thing in every toolpath—is preserved under distribution shift. The decoder preserves substantially more; the lookup preserves essentially none. This is the advantage that sensor-conditioned reconstruction brings, and it is the advantage one would expect to matter in anomaly-detection applications where the signal of interest is a structural deviation from expected grammar (command substitution, parameter swap) rather than a coordinate-level mismatch.

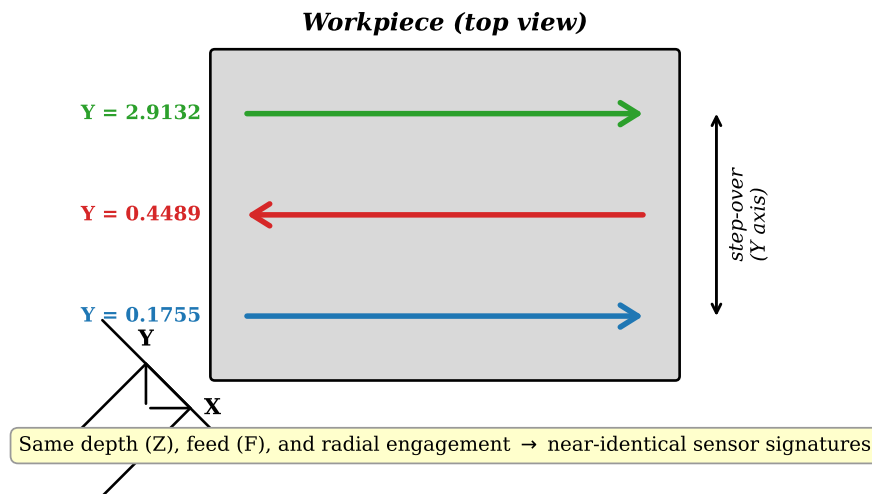
Limitations of this experiment.

The three splits are single-seed runs, not multi-seed retrains; seed variance at the structural level is unknown. The reused fold-1 encoder has seen some windows from every toolpath (through its 5-fold CV training), so the reported numbers slightly overstate what a fully held-out pipeline would achieve—we believe this slight optimism is acceptable because both methods are compared against the same biased encoder and because the decoder’s transfer advantage over the lookup table comes from the decoder itself, not the encoder. Finally, the three held-out splits differ in training-set size (240–333 windows), which confounds toolpath-specific difficulty with training-set-size effects; a full balanced held-out evaluation would require additional experiments.

## 7.6. Numeric Error Granularity

Not all sequence errors are equal. When the decoder predicts an incorrect sequence, the error is typically localized: the command and parameter tokens are correct, and only one or two numeric digit tokens differ. The most common error pattern is a single-digit substitution in the Y-axis value (e.g., predicting the digit sequence for  $Y = 0.1755$  instead of  $Y = 2.9132$ ), which changes the meaning of the coordinate but leaves the rest of the G-code line intact. Errors that span multiple token positions (e.g., predicting G2 when the target is G1 X . . . Y . . .) are rare, consistent with the 98.1% mean command accuracy.

The sequence-level confusion matrix (Figure 11) and per-head confusion matrices (Figure A6) confirm this pattern. The command and parameter heads are near-diagonal (few off-diagonal entries), while the Y-axis numeric head shows the characteristic step-over confusion pattern. This error structure has implications for anomaly detection: even when the decoder makes a reconstruction error, the *type* of error (Y-axis step-over confusion) is predictable and can be distinguished from the error patterns that a genuine attack would produce (e.g., command substitution, feed rate manipulation, or coordinate shifts along unexpected axes).



**Figure 12.** Lateral step-over passes at different Y coordinates. Each pass follows the same trajectory in the XZ plane with identical cutting conditions (depth, feed rate, radial engagement), producing sensor readings that are indistinguishable within a single observation window. Full-precision Y values shown.

The Y-axis values 0.1755, 2.9132, and 0.449 correspond to successive lateral step-over positions: each pass follows a similar trajectory in the XZ plane at a different Y coordinate. In 3-axis vertical milling, the cutting dynamics—forces, accelerations, acoustic emission—are governed primarily by the depth of cut (Z), feed rate (F), and radial engagement, not the absolute lateral position. Consequently, passes at different Y step-over positions produce sensor readings that are indistinguishable within a single observation window. At 0.635 mm depth in UHMW-PE, the step-over distance of 3.81 mm produces no measurable change in cutting force, vibration amplitude, or acoustic emission—the tool engagement geometry is identical at each Y coordinate.

Figure 12 illustrates this geometry. Within a 256-timestep window, the sensor measurements during a pass at  $Y = 0.1755$  and a pass at  $Y = 2.9132$  are statistically indistinguishable. The physical cutting conditions are identical; only the table's lateral position differs. The Y-value confusion matrix (Figure A7) confirms this pattern: the dominant off-diagonal entries correspond to Y-value pairs at nearby step-over positions. The window position feature (V7) directly addresses this by providing the decoder with the window's ordinal position within the file, which is strongly correlated with step-over pass number. The effect is dramatic: Fold 5 jumped from 52.7% (V6, no window position) to 91.4% (V7, with window position)—a 38.7-point improvement attributable primarily to Y-axis disambiguation.

The preceding sections quantified overall accuracy and identified which features and modalities matter most. This section examines where the decoder fails and why.

## 8. Error Analysis

### 8.1. Error Decomposition

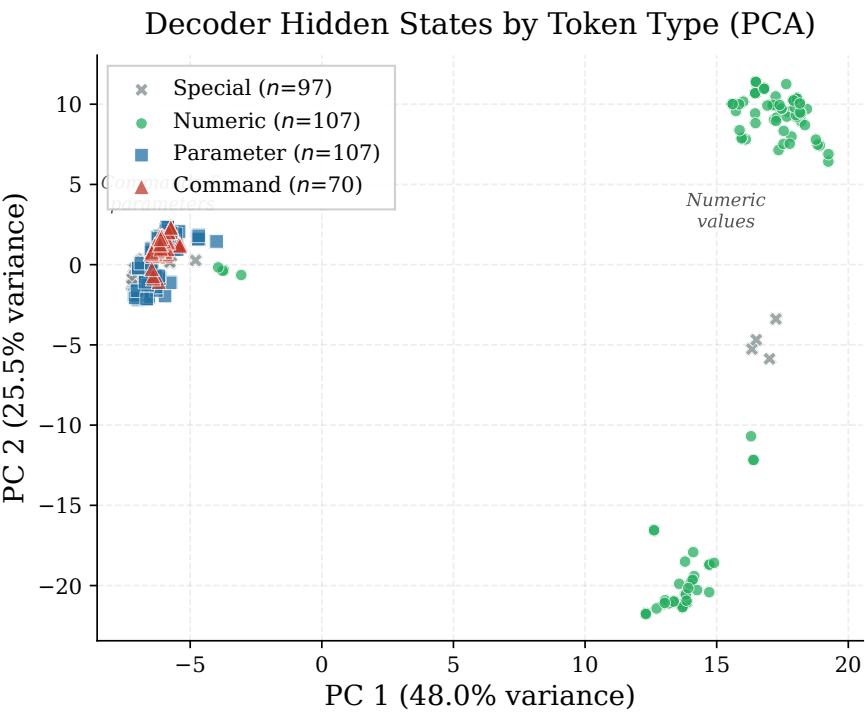
At 86.1% mean cross-fold sequence accuracy (91.4% best fold), the remaining errors decompose hierarchically:

- **Command errors: <2%.** Command accuracy averages 98.1% across folds, with two folds at 100%. The decoder rarely confuses motion types (G0/G1/G2/G3), as confirmed by the token-type confusion matrix (Figure A3).
- **Parameter type errors: <1%.** Parameter addresses (X, Y, Z, F, R) average 97.6% accuracy across folds.
- **Numeric value errors: ~8.7%.** The dominant error source. Nearly all numeric errors involve:
  - Y-axis step-over confusion: 0.1755 ↔ 2.9132 ↔ 0.449 (lateral passes with identical cutting conditions)
  - X-axis confusion: geometrically similar coordinates producing indistinguishable sensor responses

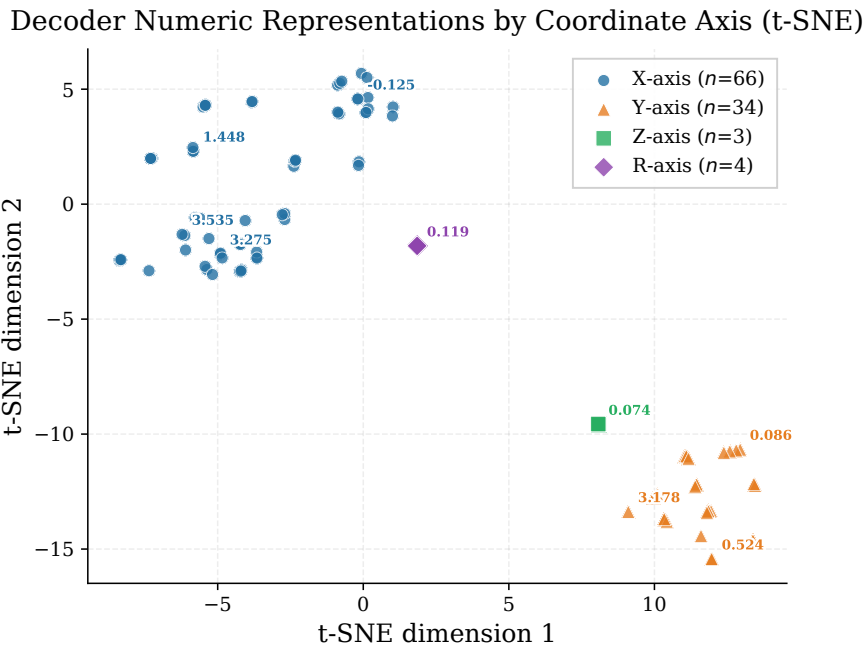
The improvement from V5 ( $75.1 \pm 10.0\%$  cross-fold mean) to V7 ( $86.1 \pm 8.4\%$ ) was driven primarily by resolving Y-axis step-over ambiguity through window position metadata, confirming the information-theoretic analysis below.

### 8.2. Decoder Representation Analysis

To understand what the decoder learns internally, Figures 13 and 14 visualize the decoder's hidden states via PCA and t-SNE. The token-type PCA (Figure 13) shows clear separation between command, parameter, and numeric tokens in the decoder's representation space, confirming that the hierarchical token decomposition is reflected in the learned representations. The numeric-token t-SNE (Figure 14) reveals clustering by coordinate axis (X, Y, Z, R values occupy distinct regions), with within-axis structure showing that similar coordinate values are nearby in the decoder's representation. This spatial organization explains why numeric errors are typically within-axis substitutions rather than cross-axis confusions.



**Figure 13.** PCA of decoder hidden states by token type (Fold 5, 93 test windows). Commands, parameters, and numeric tokens occupy distinct regions, confirming that the decoder learns type-structured representations.



**Figure 14.** t-SNE of decoder hidden states for numeric tokens only, colored by coordinate axis. X, Y, Z, and R values cluster by axis, with within-axis proximity reflecting coordinate similarity.

8.3. Per-Operation Analysis

The per-operation breakdown (Table 6) reveals systematic variation in reconstruction difficulty (Figures A4 and A5). The damaged-pocket class (60.0%) is the hardest overall, combining the geometric complexity of pocketing with the vibration noise from spindle damage. Fold 1 remains the hardest

fold at 71.2% sequence accuracy on its 132 test windows, suggesting that its specific file combination contains toolpaths with inherently ambiguous sensor signatures that neither temporal context nor window position can resolve.

#### 8.4. Information-Theoretic Analysis

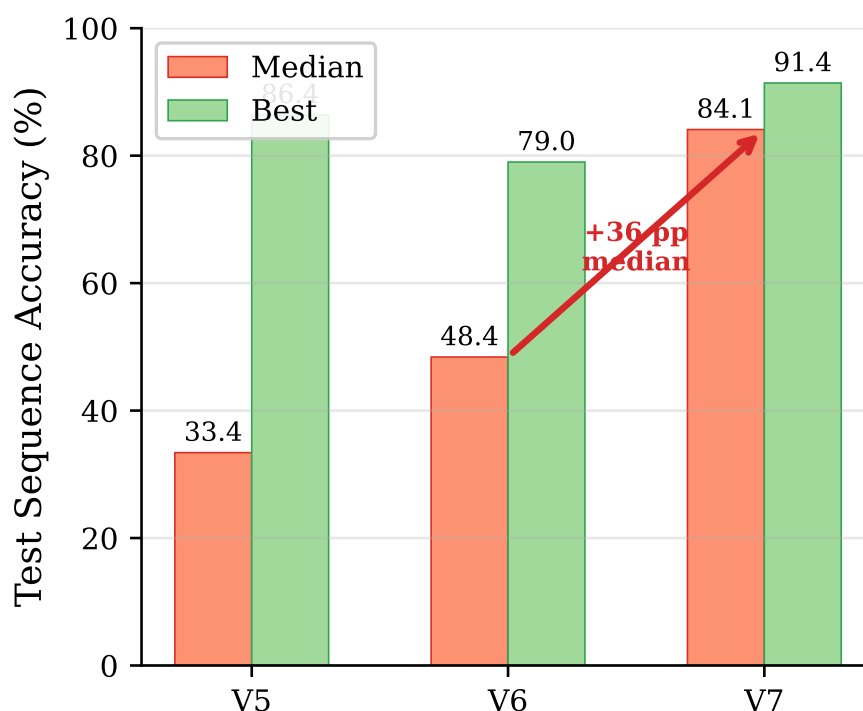
The normalized information gain of window position for Y-value prediction is  $(H(Y) - H(Y|\text{pos}))/H(Y) = (3.77 - 1.59)/3.77 = 57.8\%$ , where  $H$  denotes Shannon entropy in bits computed over training windows containing Y-axis coordinates ( $n = 656$ ). This means that knowing a window's ordinal position within the file eliminates over half the uncertainty about its Y-axis value. The relationship is strongly structured:

- Window positions 0–1:  $Y = 0.1755$  is the most frequent value ( $\sim 35\%$ )
- Window position 2:  $Y = 2.9132$  is dominant (52–93% depending on fold)
- Later positions: increasingly determined by toolpath strategy and operating condition

The structural regularity makes window position the single highest-impact feature for numeric reconstruction, consistent with V7's design decision to always enable it.

The error analysis reveals a clear hierarchy: structural prediction is solved, numeric prediction is limited by sensor physics, and window position metadata resolves most of the remaining ambiguity. The following section discusses broader implications for manufacturing, deployment, and future research.

## 9. Discussion



**Figure 15.** Data pipeline impact: median vs. best sequence accuracy across V5–V7. The +36 pp median improvement from V6 to V7 demonstrates that data-handling correctness dominates architectural improvements.

### 9.1. Why Is Command Prediction Solved but Coordinate Prediction Not?

Figure 15 summarizes the dominance of data handling over architecture. The structural component of G-code—commands and parameter addresses—is determined by toolpath geometry.

Adaptive clearing follows trochoidal paths with frequent direction reversals; face milling sweeps in linear passes with periodic step-overs; pocket machining alternates between plunges and lateral clearing. Each strategy produces distinctive acceleration profiles and trajectory envelopes that the encoder captures reliably, reaching 98.3% command accuracy in-distribution and still preserving 78–89% command accuracy on two of the three held-out-toolpath splits (Section 7.5). This is a meaningful form of transfer: the motion-command grammar is partially toolpath-invariant.

The numeric component—coordinate values—depends on the table’s absolute lateral position. In 3-axis vertical milling, the cutting dynamics (forces, vibration, acoustic emission) are governed by depth of cut, feed rate, and radial engagement, not by the table’s Y-coordinate. Two passes at  $Y = 0.1755$  and  $Y = 2.9132$  produce sensor readings that are statistically indistinguishable within a single 64-second window. This is a physical limitation, not a modeling choice, and it has a precise quantitative signature: with window-position metadata withheld, sequence accuracy drops from 86.1% to 62.3%—a 23.8 pp gap that directly measures how much of a G-code line is recoverable from sensor physics alone. The remaining numeric errors (after window position is added) concentrate on Y-values within the step-over confusion radius, and on held-out toolpaths numeric accuracy collapses to zero because unseen coordinate values are not in the closed vocabulary at all.

The structural–numeric split mirrors the separation principle in [5] and has a practical consequence for sensor-based process verification: structural anomalies (command injection, parameter substitution) are characterized at >97% in-distribution accuracy, while coordinate-level tampering would be detectable only when the deviation exceeds the step-over confusion radius of 2.74 mm.

## 9.2. Where Is the Accuracy Ceiling and What Sets It?

The encoder compresses 64 seconds of 110-channel sensor data into a 256-dimensional memory. This compression preserves toolpath-discriminative features—direction changes, engagement patterns, force profiles—but discards absolute positioning information that varies only across windows, not within them. Five observations converge on the conclusion that this encoder representation, not decoder capacity or training procedure, sets the accuracy ceiling:

1. **Fold variance exceeds every architectural effect.** The fold spread (69–90%) is larger than any architectural ablation (−2.1 pp for the sensor prior, the largest).
2. **Step-over confusion is physics-limited.** Successive lateral passes produce statistically indistinguishable sensor profiles; disambiguation requires context external to the window.
3. **External context dominates the numeric ambiguity.** Window position accounts for 23.8 of the 24.5 pp total data-feature ablation effect, with a normalized information gain of 57.8% for Y-value disambiguation.
4. **The decoder is not overfitting and is overparameterized.** Mean final train-val token-accuracy gap across 25 multi-seed runs is 2.5%, and a 3.0M-parameter decoder (2 layers,  $d=192$ ) achieves 84.4%—only 1.6 pp below the full 22.1M-parameter architecture. Capacity is not the constraint.
5. **Decoding search is not the constraint either.** Teacher-forced, greedy autoregressive, and beam-search decoding produce numerically identical aggregate accuracies; per-item predictions differ on only 7.1% of windows (18% on the hardest fold), and those differences are symmetric—decode modes swap which windows they get right, but the number of correct predictions is constant. Errors are representational, not search errors.

Preliminary fine-tuning experiments (unfreezing the encoder LSTM) improved Fold 1 by only 2.3 pp, consistent with the representation being near-optimal for the available modalities and window size. Architectures that attend across multiple windows at the encoder level—rather than concatenating independently encoded memories as the current decoder does—might capture inter-pass dynamics the current single-window encoder cannot. Whether such an architecture would *break* the physics ceiling or merely *approach* it more efficiently is an open question: the held-out-toolpath result (Section 7.5) suggests that some component of numeric reconstruction requires genuinely new modeling assumptions (e.g., open-vocabulary digit generation) rather than just a bigger architecture.

### 9.3. What Might Reconstruction Enable for Physical-Layer Verification?

We stress that the following is a characterization of what reconstruction accuracy implies in principle, not a validated anomaly-detection result. The reconstruction framework could in principle serve as an independent physical verification channel: comparing sensor-predicted G-code against commanded G-code would reveal discrepancies from injection attacks, parameter modification, or controller compromise [25,26]. The error structure from Section 8 bounds what would be detectable and what would not. Command-level injections (substituting G0 for G1, injecting G2 arcs) would be flagged in nearly all cases given 98.3% in-distribution command accuracy and 78–89% command accuracy even on held-out toolpaths (Section 7.5). Parameter substitution (swapping X for Y addresses) is characterized at 97.2% in-distribution parameter accuracy. Coordinate-level tampering, however, would be detectable only when the shift exceeds the Y-axis step-over confusion radius of 2.74 mm ( $|2.9132 - 0.1755|$ , the dominant confusion pair)—and this radius is three orders of magnitude larger than typical precision-machining tolerances ( $\pm 0.025$ – $0.050$  mm). A subtle attacker could displace coordinates within this radius without detection. Sensor-based reconstruction is therefore better suited to detecting grammar-breaking attacks (novel commands, parameter swaps) than to detecting quantitative coordinate drift. End-to-end validation of these detection capabilities—synthetic command injection, parameter substitution, leave-one-class-out novelty detection, entropy-based scoring, and physical attack experiments—is the subject of a separate companion paper [6] and is not claimed here.

### 9.4. What Does Deployment Require?

**Sensor configuration.** The modality ablation (Table 11) provides direct guidance. Gyroscope, color, and accelerometer are the three most critical modalities (−23 to −28 pp each when removed). Audio contributes the least (−6.5 pp). A minimal deployment using only IMU boards (gyroscope + accelerometer), color sensors, and electrical monitoring retains the top-4 modalities at an estimated hardware cost of ~\$492 [1], with no controller modification required.

**Observation window.** Halving from 64 s ( $W=256$ ) to 32 s ( $W=128$ ) costs 9.2 pp (76.9% accuracy) but doubles temporal resolution. A 16 s window ( $W=64$ ) retains 70.6% at  $4\times$  resolution. The choice depends on whether the application prioritizes accuracy (process verification) or response time (real-time anomaly detection).

**Compute.** Greedy decoding averages 19.9 ms per window on an RTX A6000, well within the 64-second acquisition period. The capacity ablation shows that a 3.0M-parameter decoder (2 layers,  $d=192$ ) achieves 84.4%, suggesting sub-10 ms edge deployment is feasible. Beam search ( $k \in \{1, 3, 5\}$ ) produced identical accuracy to greedy decoding on all folds, confirming that errors are representational, not search errors.

**Vocabulary.** The 712-token vocabulary is geometry-specific, and in practice only 48 tokens (14 structural + 34 numeric) are actually exercised by this dataset's 34 unique G-code lines (Appendix C.4). New parts require adding numeric tokens for unseen coordinates. The structural subset covers the motion commands and parameter letters used in basic 3-axis linear and circular interpolation; richer G-code dialects with canned drilling cycles (G81–G89), cutter compensation (G41/G42), coordinate-system switches (G54–G59), or subroutine calls would need vocabulary extensions not tested here. The digit-by-digit prediction framework [5] could eliminate the closed-vocabulary limitation entirely.

### 9.5. Limitations

#### 1. Single workpiece, single machine, single material, single tool, single set of cutting parameters.

All 120 files machine the same part, on one Bantam Tools Explorer™ desktop CNC mill, using a 2-flute 1/4" HSS endmill in TIVAR UHMW-PE at fixed feed and speed. This is an intentional experimental control for a proof-of-concept study, but it narrows the external

validity of every quantitative result in the paper. The decoder architecture, training procedure, grammar constraints, and evaluation methodology transfer directly to new setups; the vocabulary, window-position-to-coordinate mapping, encoder weights, and modality importance ranking are specific to this configuration and would need to be relearned.

2. **Desktop vs industrial machines.** Industrial CNC machines differ from hobby-grade desktop mills in structural stiffness, vibration modes (1–5 Hz frame resonances vs. 10–50 Hz on desktop), tool holder acoustics, coolant systems, spindle speeds, and axis counts. Force magnitudes on industrial machines may saturate low-cost MEMS accelerometers. None of the sensor placement, modality importance, or accuracy results in this paper should be assumed to transfer to industrial deployments without direct evaluation.
3. **Polymer vs metal cutting.** UHMW-PE was chosen for reproducible, low-variation cutting forces. Metals generate fundamentally different force, thermal, and acoustic signatures; continuous chip formation, built-up edge, tool wear, and thermal drift all change within-operation dynamics. The modality importance ranking we report (gyroscope and color dominating, audio least important) is material-specific and may not survive in metal cutting, where audio and acoustic-emission signals typically carry substantially more information.
4. **3-axis vocabulary, limited G-code dialect.** The structural vocabulary captures only basic 3-axis linear and circular interpolation (G0–G3 with X/Y/Z/F/R). Canned drilling cycles (G81–G89), cutter compensation (G41/G42), coordinate-system switches (G54–G59), subroutine calls, and 4/5-axis commands are not represented. Real production G-code routinely uses these constructs.
5. **Small, long-tailed dataset.** 545 windows, 34 unique G-code sequences (17 with fewer than 20 windows), 5 folds. The statistical power of the ablation ANOVA is limited (window position is the only ablation to reach significance after Bonferroni correction); the Wilcoxon tests on sweep progression do not reach  $\alpha = 0.05$  at  $n = 5$  folds even when all differences are in the favorable direction. The held-out-toolpath experiments are single-seed per split. Larger, more diverse datasets—with more unique G-code sequences and more files per operation class—are needed before any of these numbers should be treated as stable estimates.
6. **Closed vocabulary.** Coordinates not in the 712-token vocabulary cannot be reconstructed, which is why the held-out-toolpath experiments collapse on numeric accuracy. The digit-by-digit framework [5] could enable open-vocabulary prediction but has not yet been integrated with this decoder.
7. **Temperature as a retained environmental channel.** The f98 “confound-free” encoder removes proximity and pressure but retains temperature, which is also an environmental variable subject to ambient drift. We have not run the f97 (no-temperature) ablation that would quantify how much of the f98 accuracy depends on temperature.
8. **Single simulated fault mode.** The “damaged” condition is air-cutting with one spindle drive band removed—a specific simulated runout. Real spindle-health issues include bearing wear, unbalance, and thermal growth, whose sensor signatures differ from this particular fault.
9. **Anomaly-detection claims analytical only.** Detection capabilities are inferred from reconstruction accuracy and error structure, not validated on actual or simulated attacks. End-to-end validation is the subject of a separate companion paper [6] and is not claimed here.
10. **Held-out-toolpath encoder leakage.** The held-out-toolpath experiments in Section 7.5 reuse the fold-1 V7 encoder, which saw some files from every toolpath class during its original 5-fold CV training. A stricter held-out-toolpath evaluation would train fresh encoders with the heldout class excluded; we did not do this, so the reported numbers are a mild upper bound on truly held-out transfer performance.

## 9.6. Future Work

- **Anomaly detection validation:** synthetic G-code injection, leave-one-class-out novelty detection, entropy-based scoring, and physical attack experiments [6].

- **Open-vocabulary numerics:** integrating digit-by-digit generation from [5] to eliminate the closed-vocabulary limitation.
- **Multi-machine generalization:** testing on industrial machines with different dynamics, sensor configurations, materials, and G-code dialects.
- **Cross-window encoder:** attending across multiple windows at the encoder level to capture inter-pass dynamics the current single-window architecture misses.
- **End-to-end fine-tuning:** backpropagating the decoder’s reconstruction loss through the encoder, with careful regularization given the 545-sample dataset.
- **LSTM decoder comparison.** We did not run a matched-capacity LSTM decoder baseline in this work, and therefore do not claim architectural superiority of the Transformer decoder over an LSTM of similar parameter count. The SensorMultiHeadDecoder architecture is evaluated against lookup, retrieval, tree-based, and MLP-classifier baselines (Table 8), but not against a recurrent alternative. A matched LSTM-decoder ablation is a natural next step.

## 10. Conclusion

This paper presented a sensor-to-G-code reconstruction system that autoregressively generates CNC machine instructions from multi-modal sensor embeddings, and used it to quantify what is and is not recoverable from physical measurements on a controlled desktop milling dataset (120 files, 545 windows, 9 operation classes, single material). Through seven progressive hyperparameter sweeps totaling 4,782 cross-validated experiments, seven baseline comparisons including a zero-parameter position lookup, a 12-condition ablation study, a 9-modality sensor ablation, a window-size ablation, and three leave-one-toolpath-out held-out evaluations, we report 84.5% mean cross-fold sequence accuracy (single-seed per fold average; best-of-5-seeds retrain 86.1%), 94.6% token accuracy, and 98.3% command accuracy.

The key findings are:

1. **Sensor-to-G-code reconstruction has a physical ceiling.** A decoder given only sensor embeddings (no within-file position metadata) reaches 62.3% sequence accuracy. Adding one ordinal position feature lifts this to 86.1%. The 23.8 pp gap directly quantifies how much of a G-code program is recoverable from sensor physics alone versus from file structure, and is concentrated on Y-axis lateral step-over coordinates where successive passes produce sensor dynamics that are statistically indistinguishable within one observation window. No architectural feature of the decoder exceeds 2.1 pp individual contribution.
2. **A parameterless lookup table outperforms the decoder on the fixed geometry** (90.4% vs 84.5% mean-of-seeds), confirming that on a closed dataset where (source\_file, window\_index) is nearly a unique key, the dominant signal is metadata, not sensors. The decoder’s value is therefore not raw accuracy on the training geometry but what it preserves under distribution shift.
3. **In held-out-toolpath evaluation, both methods produce 0% exact-sequence accuracy** because held-out toolpath classes use numeric coordinate values absent from training, but the decoder preserves structural grammar (command and parameter-type accuracy) substantially better than the lookup table—by 27 pp and 53 pp respectively, averaged across three held-out splits. Structural transfer is real, numeric transfer (under a closed vocabulary) is not.
4. **Sensor importance reverses between tasks.** Gyroscope (−27.7 pp) and the APDS-9960 color channel (−27.2 pp) are the two most critical modalities for reconstruction, while audio—the second-strongest classifier feature—is the least important (−6.5 pp). The natural explanation for the color channel’s importance—ambient-light correlation with Y-axis position—was tested empirically and refuted (max within-file  $|r| = 0.12$ ); the mechanism remains unexplained.
5. **The decoder is not overfitting.** Across 25 multi-seed V7 runs the mean final train-val token-accuracy gap is 2.5% (median 1.4%), which for a 22.1M-parameter model on 545 samples indicates that the binding constraint is the encoder’s sensor representation, not the decoder’s capacity. A 3.0M-parameter decoder reaches 84.4%.

**6. Reconstruction degrades gracefully under resource reduction.** Removing barometrically confounded proximity/pressure channels drops accuracy to 82.5%. Halving the observation window from 64 s to 32 s drops accuracy to 76.9% at  $1.6\times$  temporal resolution. Structural (command + parameter) accuracy stays above 95% in all reduced configurations.

**Practical recommendations for future deployment studies.** These are deployment targets, not deployment results; all recommendations below are conditional on reproducing the reconstruction pipeline on different hardware, materials, and tools. For maximum reconstruction fidelity, deploy the full sensor suite with the 256-timestep (64 s) observation window. For cost-constrained setups, gyroscope, accelerometer, color, and electrical channels (the top-4 modalities) are the ones to keep first. For applications that value temporal responsiveness over exact numeric recovery, the 128-timestep (32 s) window remains viable for structural monitoring. A 3.0M-parameter decoder is sufficient on the edge. Sensor selection for G-code reconstruction must differ from selection for operation-class monitoring—the two tasks weight modalities in nearly opposite order.

**Relation to physical-layer anomaly detection.** The structured error patterns documented in Section 8 and the held-out-toolpath results in Section 7.5 together characterize what a sensor-conditioned G-code reconstructor can and cannot detect: structural deviations (wrong motion command, swapped parameter address) are recoverable at 97%+ in-distribution and 78–89% under toolpath shift, while coordinate-level tampering is detectable only when the deviation exceeds the Y-axis step-over confusion radius of 2.74 mm. This suggests that sensor-based reconstruction is better suited to detecting grammar-breaking attacks than to detecting subtle coordinate perturbations. We defer the anomaly-detection experiments—synthetic command injection, parameter substitution, leave-one-class-out novelty detection, entropy-based scoring, and physical attack simulation—to a separate companion paper [6]. This work establishes only what is physically recoverable from sensor data; it does not claim that such recovery has been validated as an attack detector.

**Author Contributions:** Conceptualization, S.S.E. and R.S.P.; methodology, S.S.E.; software, S.S.E.; validation, S.S.E.; formal analysis, S.S.E.; investigation, S.S.E.; resources, M.S.S.; data curation, S.S.E. and R.S.P.; writing—original draft, S.S.E.; writing—review & editing, R.S.P. and M.S.S.; visualization, S.S.E.; supervision, R.S.P. and M.S.S.; project administration, R.S.P. All authors have read and agreed to the published version of the manuscript.

**Funding:** This material is based upon work partially supported by the Office of Naval Research under Contract no. N000142412129.

**Acknowledgments:** Any opinions, findings, and conclusions or recommendations expressed in this material are those of the authors and do not necessarily reflect the views of the Office of Naval Research. The authors thank the Department of Industrial Systems Engineering at the University of Rhode Island for providing laboratory facilities and computational resources.

**Conflicts of Interest:** The authors declare no conflict of interest.

**Data Availability Statement:** The sensor datasets and analysis code generated during this study are available at [https://github.com/stephenseacuello/gcode\\_fingerprinting](https://github.com/stephenseacuello/gcode_fingerprinting).

**Software Availability Statement:** Source code for the SensorMultiHeadDecoder architecture, experiment pipeline, and analysis scripts is available at [https://github.com/stephenseacuello/gcode\\_fingerprinting](https://github.com/stephenseacuello/gcode_fingerprinting) under the MIT License.

## Appendix A Frozen Encoder: MM-DTAE-LSTM Architecture and Training

This appendix describes the frozen encoder that produces the sensor memory the decoder attends to. It is provided so that this paper can stand alone without requiring access to the under-review encoder paper [1]; the architectural details below are reproduced from that source.

Input representation.

Each window is a tensor  $\mathbf{X} \in \mathbb{R}^{256 \times 110}$ : 256 aligned timesteps  $\times$  110 continuous sensor features (Table 2). Four additional categorical features (timestamp-derived phase indicators) are carried alongside but are not processed by the encoder.

Modality-specific input projections.

The 110 continuous features are partitioned by modality into seven groups: accelerometer (18 channels), gyroscope (18), magnetometer (18), pressure (6), color RGBA (24), audio RMS (6), and electrical (8). The 4 environmental channels (temperature, proximity) are grouped with pressure for a total of 18 environmental features in the f110 configuration. Each modality group is independently projected to a 256-dimensional latent via a small fully connected block (Linear  $\rightarrow$  LayerNorm  $\rightarrow$  GELU  $\rightarrow$  Dropout). This gives each modality an equal-dimensional representation before fusion, and prevents high-count modalities (color, IMU triplets) from dominating the attention computation by raw dimensionality.

Cross-modal attention gating.

The seven per-modality 256-dim embeddings are stacked along a “modality” axis and passed through a multi-head self-attention block. This block learns soft gates that allow each modality to attend to every other modality at every timestep, implementing the paper’s claim that modalities are not treated as independent feature channels but as interacting streams. Gating weights are timestep-specific, so (for example) the color channel can receive more weight during direction changes than during steady-state passes.

Denoising Temporal Attention Encoder (DTAE).

The fused stream is passed through a Transformer encoder block with a denoising reconstruction objective during pretraining: a fraction of input channels is randomly masked, and the encoder is trained to reconstruct the masked channels from the remaining context. This provides a self-supervised signal that shapes the encoder’s representation toward robust multi-modal fusion before the classification objective is introduced.

Bidirectional LSTM temporal model.

The DTAE output is processed by a 2-layer bidirectional LSTM with 256 hidden units per direction. The forward and backward states are concatenated and projected back to 256 dimensions, producing the final memory tensor  $\mathbf{M} \in \mathbb{R}^{L \times 256}$ , where  $L$  is a compressed sequence length after temporal pooling. This memory is what the decoder attends to via cross-attention.

Auxiliary classification head.

During encoder pretraining, a 9-way softmax head on the mean-pooled memory supervises operation-class classification (see Table 3). This head is discarded when the encoder is frozen for decoder training; only the memory  $\mathbf{M}$  is used downstream.

Parameter count and feature configurations.

The full f110 encoder has approximately 5.1 million parameters: modality projections  $\sim 0.3\text{M}$ , cross-modal attention  $0.8\text{M}$ , DTAE  $1.6\text{M}$ , bidirectional LSTM  $2.4\text{M}$ . The variant encoders used in Section 7.3.1 (f98 no-proximity-no-pressure, f104 no-proximity, etc.) have the same architecture with the modality-specific projection input dimensions reduced accordingly, keeping the post-projection 256-dim bottleneck unchanged.

Pretraining objective.

The encoder is trained with a three-term loss: the 9-class cross-entropy classification objective, a denoising reconstruction loss on the masked channels, and a contrastive loss that pushes embeddings of different operation classes apart in the pooled-memory space. The three terms are balanced with fixed weights determined in an earlier sweep and are not tuned as part of this paper’s decoder work. All encoder training, ablation, and sensor-importance analysis—including the 9-class ANOVA-based

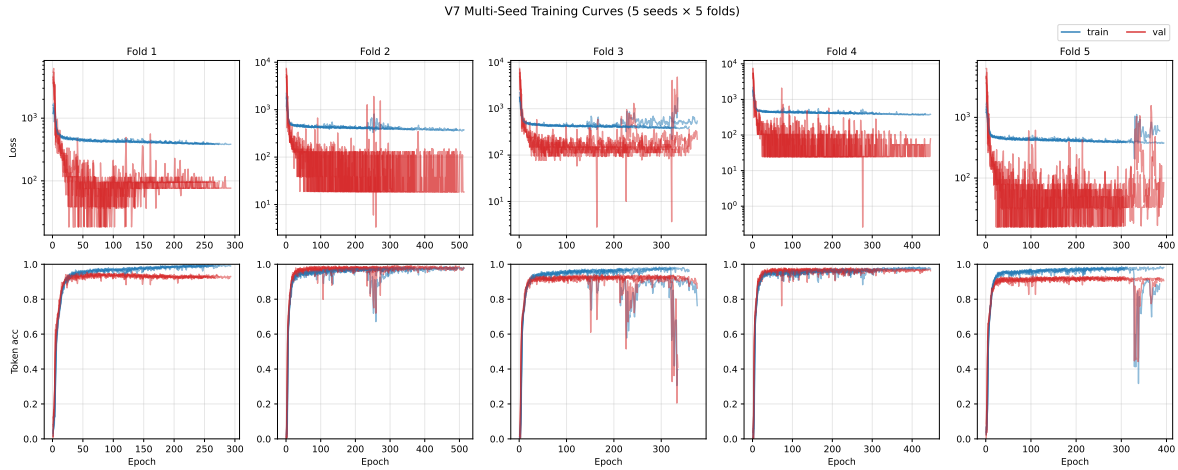
feature rankings cited in Table 11—is reported in detail in [1]; only the final frozen checkpoint is consumed here.

What the decoder sees.

For every input window  $\mathbf{X}$ , the frozen encoder produces  $\mathbf{M} \in \mathbb{R}^{L \times 256}$ . The decoder’s cross-attention treats  $\mathbf{M}$  as the memory bank and autoregressively emits G-code tokens conditioned on it. Neither the encoder weights nor any of the encoder’s intermediate activations (modality gates, LSTM states, etc.) are modified during decoder training. This design isolates decoder-level effects from encoder-level changes, at the cost of inheriting whatever representational limits the encoder imposes—limits that, per Sections 7.2 and 7.6, are the dominant factor in the decoder’s remaining errors.

## Appendix B Supplementary Figures

The following figures provide additional detail supporting the analyses in the main text.



**Figure A1.** Training and validation curves for all 25 V7 multi-seed runs (5 folds  $\times$  5 seeds). Top row: log-scale loss. Bottom row: token accuracy. Train curves in blue, val curves in red; each panel overlays the 5 seeds for that fold. Fold 1 (leftmost) shows the largest train-val gap at  $\sim 6\%$ ; all other folds show gaps below  $1.5\%$ . Mean final train-val token-accuracy gap across all 25 runs:  $2.5\%$  (median  $1.4\%$ ). This near-zero gap on a 22.1M-parameter decoder trained on 545 windows is the primary evidence that the binding constraint is the encoder’s sensor representation, not decoder capacity.

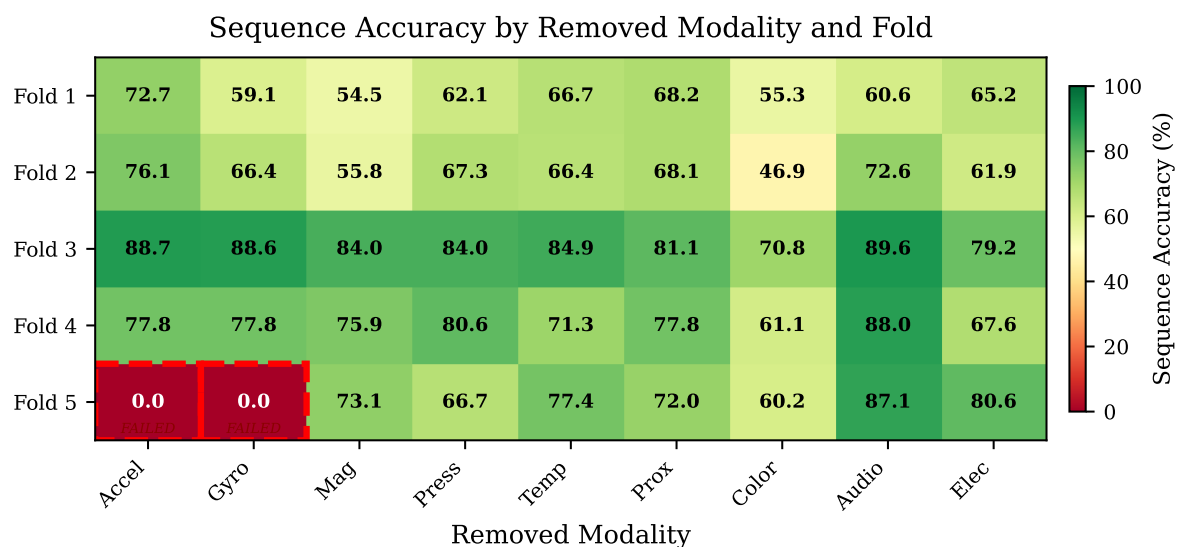
## Appendix C Sweep Details and Supplementary Feature Analyses

This appendix provides the detailed sweep-by-sweep narrative and feature analyses that informed the final V7 configuration presented in the main text.

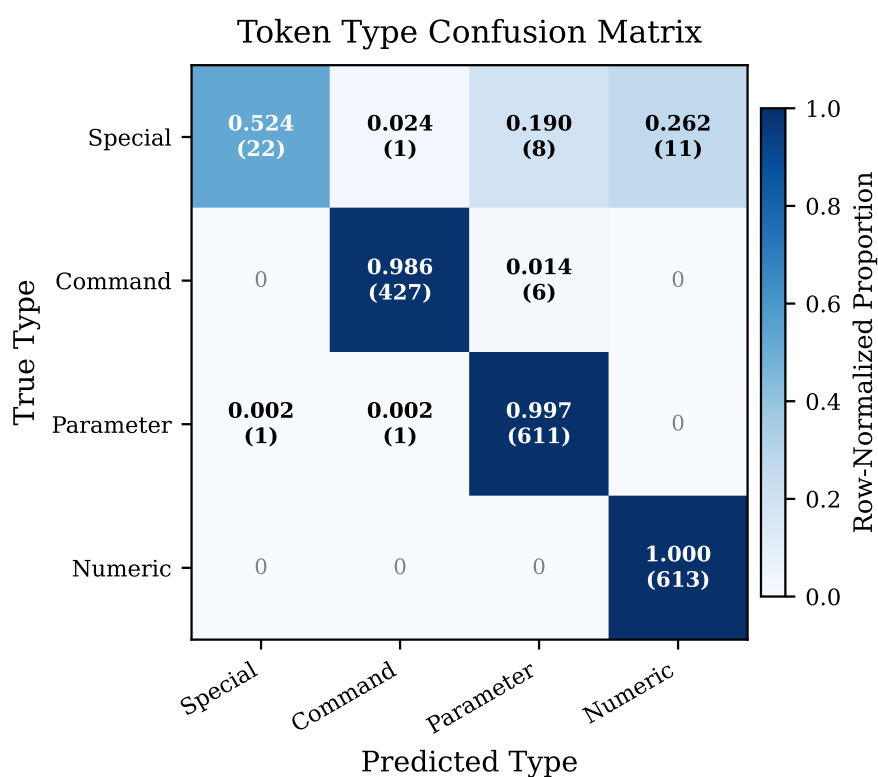
### Appendix C.1 V1–V7 Phase Narrative

**Phase 1: Training sufficiency (V1→V3, +16.3 pp).** The jump from 59.9% to 76.2% was primarily driven by longer training (100→500 epochs) allowing deeper models ( $n_{\text{layers}} = 8$ ) to converge, and cross-validation revealing that single-fold evaluation was both optimistic for some folds and pessimistic for others.

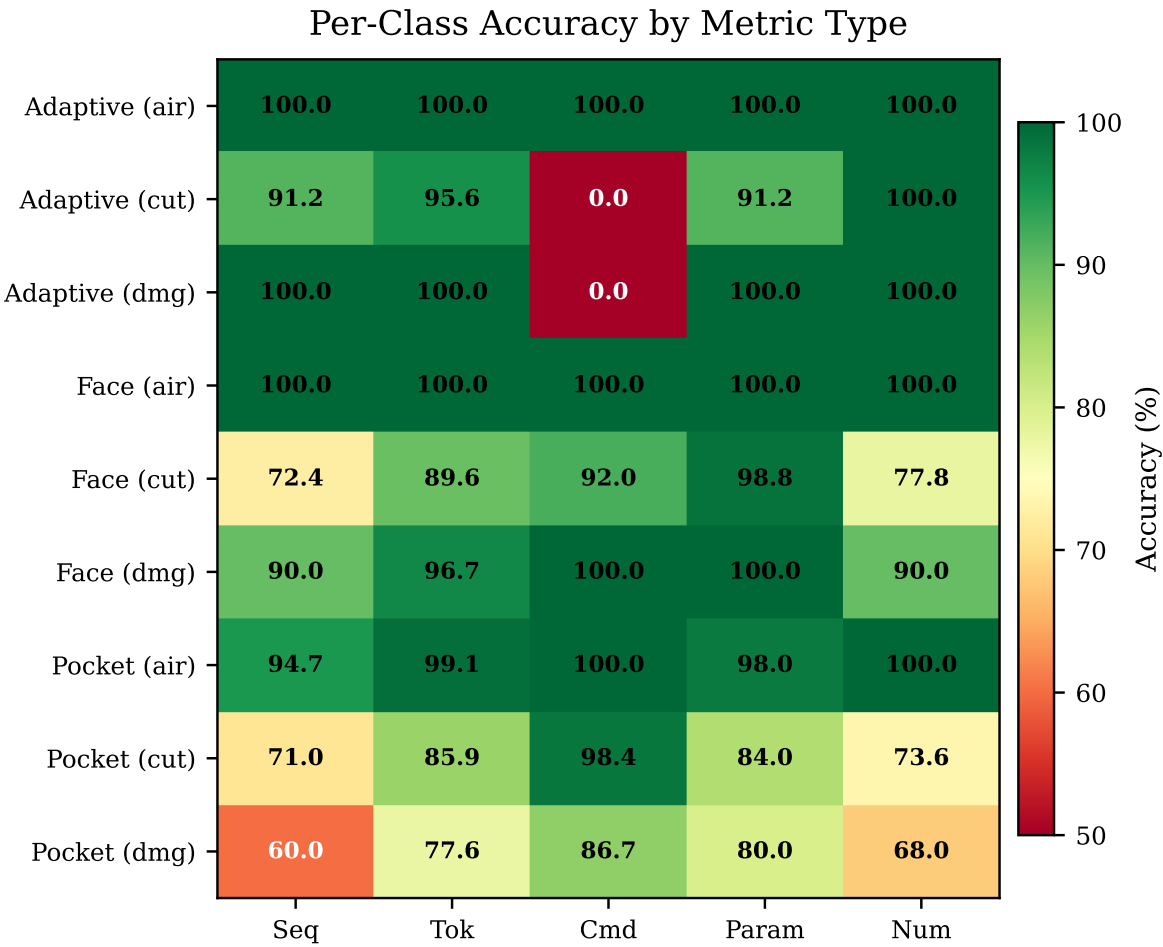
**Phase 2: Temporal context (V3→V5, +10.2 pp).** Memory positional encoding and multi-window context provide the largest architectural improvement. In V5 (before window position was available), the combination achieves Cohen’s  $d = 1.09$  against the no-context baseline—a large effect size. Note that this  $d$  was measured with V5’s buggy random-neighbor MWC; the V7 ablation (Table 9) shows the corrected MWC contributes only  $-2.8$  pp once window position is present.



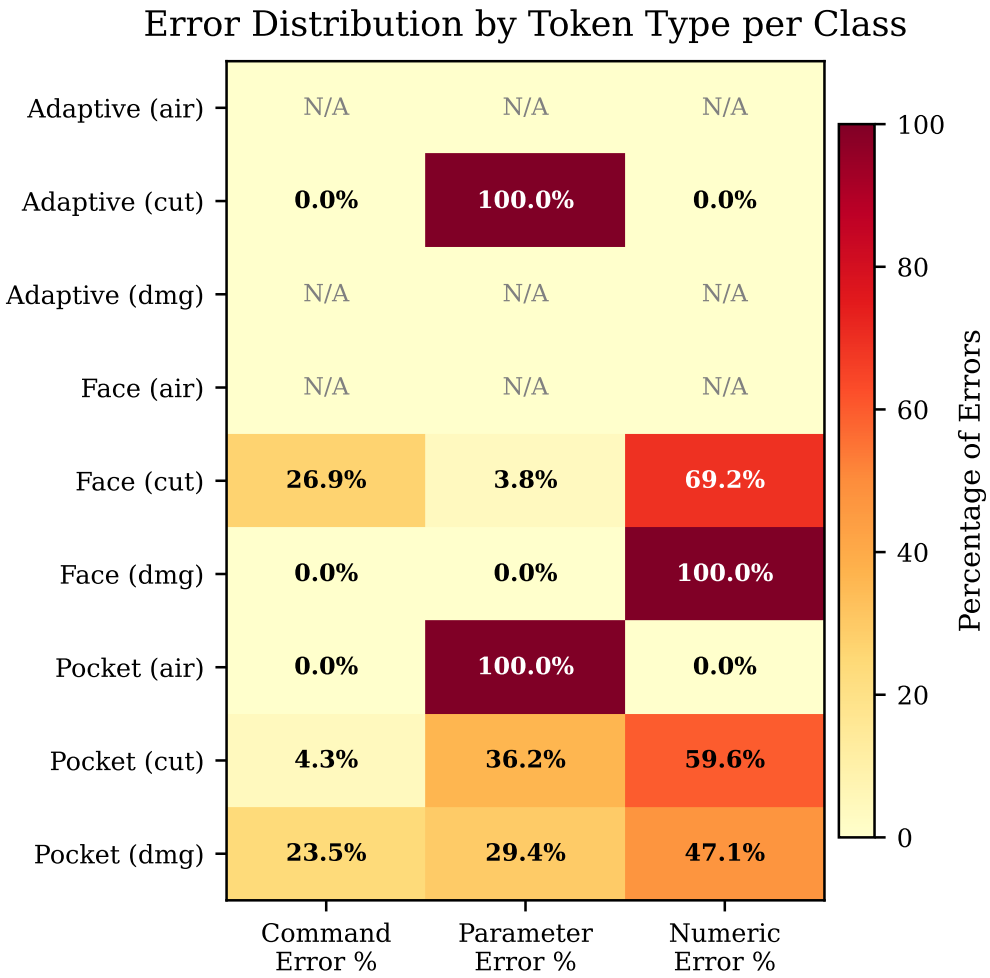
**Figure A2.** Fold  $\times$  modality interaction. Two catastrophic failures (Fold 5 without gyroscope or accelerometer) indicate degenerate encoder representations for specific fold/modality combinations. Audio removal shows the smallest and most uniform degradation.



**Figure A3.** Token-type confusion matrix (all 552 test windows). Numeric tokens are classified with perfect type accuracy (1.000). Command tokens occasionally scatter to Parameter (0.014). The rare Special  $\rightarrow$  Numeric confusion (0.262) occurs at sequence boundaries where EOS is predicted late.

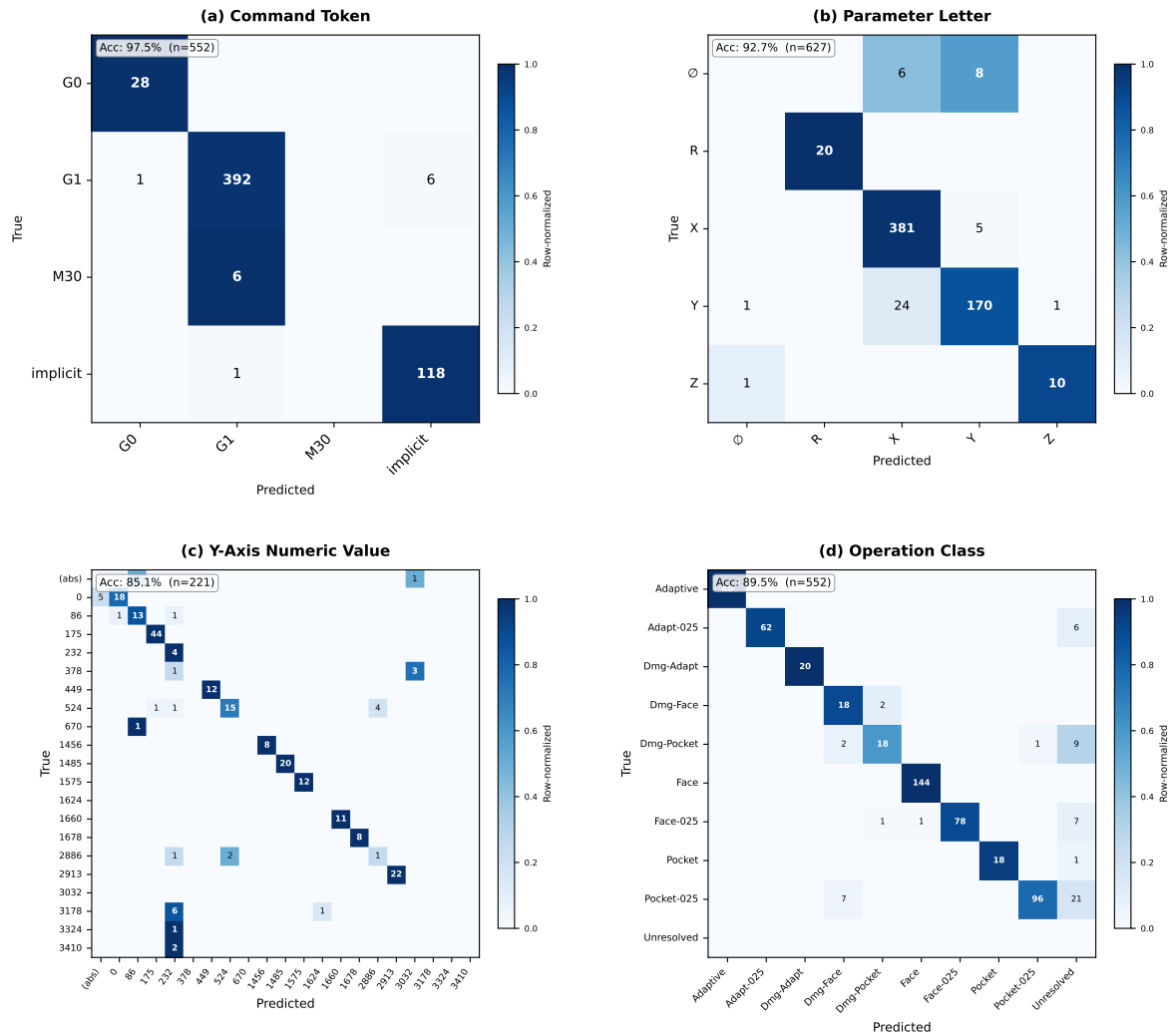


**Figure A4.** Per-class × metric heatmap. Air-cutting classes achieve 100% across all metrics. Degradation in active-cutting and damaged-spindle classes concentrates in sequence and numeric accuracy. Cells showing “n/a” for command accuracy reflect modal persistence: the G-code line contains no explicit command token, so the metric is undefined for that class.

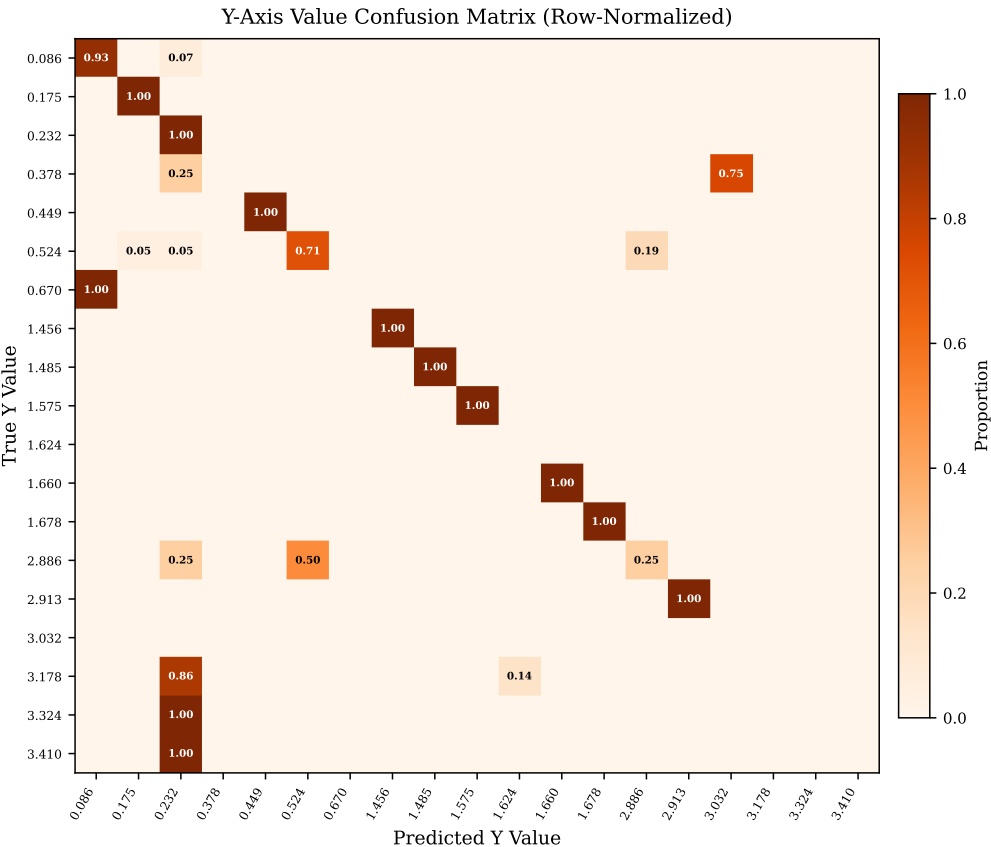


**Figure A5.** Error type distribution per class (classes with zero errors omitted). Numeric errors dominate face and pocket classes; parameter errors dominate adaptive active-cut.

**Per-Prediction-Head Confusion Matrices  
(552 test windows, 5-fold cross-validation)**



**Figure A6.** Per-head confusion matrices across all 552 test windows (V7 best config, 5 folds). **(a)** Command tokens: 97.5% accuracy; M30 is the only systematically misclassified command. **(b)** Parameter letters: 92.7% accuracy; the dominant error is Y predicted as X (24 cases). **(c)** Y-axis numeric values: 85.1% accuracy—the hardest head, confirming that Y-axis step-over confusion is the reconstruction bottleneck. **(d)** Operation class: 89.5% of predictions resolve to a known target sequence; pocket machining under active cutting contributes the most errors.



**Figure A7.** Y-axis numeric value confusion matrix (row-normalized). Most Y-values are predicted correctly (strong diagonal). The primary confusion pattern is between step-over values at similar lateral positions:  $Y=0.378$  is frequently predicted as  $Y=3.324$ , and  $Y=0.524$  splits across  $Y=0.086$  and  $Y=0.175$ . These confusions correspond to passes at nearby Y-coordinates where the measured dynamics are nearly identical.

**Phase 3: Data pipeline correctness (V5→V7, +5.0 pp best, +36 pp median).** V6 regressed to 79.0% because three high-impact data features were structurally blocked by missing metadata in preprocessed files. V7 re-processed data from raw CSVs, unblocking all three. The result is the largest improvement in *robustness*: the median sequence accuracy rose from 33% (V5) and 48% (V6) to 84% (V7), meaning that nearly every hyperparameter configuration now produces a viable model.

#### Appendix C.2 Best Test Sequence Accuracy by Fold

**Table A1.** Best Test Sequence Accuracy (%) by Fold

|            | Fold 1      | Fold 2      | Fold 3      | Fold 4      | Fold 5      | Spread      |
|------------|-------------|-------------|-------------|-------------|-------------|-------------|
| V5         | 63.6        | 86.4        | 81.0        | 79.0        | 65.6        | 22.8        |
| V6         | 59.8        | 72.7        | 79.0        | 65.7        | 52.7        | 26.3        |
| <b>V7*</b> | <b>73.5</b> | <b>89.4</b> | <b>86.8</b> | <b>88.9</b> | <b>91.4</b> | <b>17.9</b> |

\*V7 row: best trial across all 157 sweep configurations per fold.

Table 5 reports the single best config retrained with multiple seeds (slightly different per-fold values due to seed variance).

#### Appendix C.3 V7 Sweep Metrics

Table A2 summarizes the final results across all 157 completed V7 trials.

**Table A2.** V7 Test-Set Metrics (157 trials, best per metric across all folds)

| Metric           | Best         | Median | Mean | Min  |
|------------------|--------------|--------|------|------|
| Sequence acc.    | <b>91.4</b>  | 84.1   | 81.5 | 63.6 |
| Token acc.       | <b>97.9</b>  | 95.2   | 93.8 | 84.6 |
| Numeric acc.     | <b>95.7</b>  | 92.3   | 89.9 | 76.6 |
| Command acc.     | <b>100.0</b> | 97.7   | 98.2 | 82.4 |
| Param. type acc. | <b>99.2</b>  | 97.2   | 96.6 | 91.5 |

#### Appendix C.4 Converged Hyperparameters

**Table A3.** Converged Hyperparameters. Values locked after the indicated sweep and fixed for all subsequent experiments.

| Parameter          | Value              | Locked at |
|--------------------|--------------------|-----------|
| $d_{\text{model}}$ | 384                | V2        |
| Learning rate      | $3 \times 10^{-4}$ | V6        |
| Batch size         | 32                 | V2        |
| Dropout            | 0.1                | V2        |
| Focal $\gamma$     | 2.0                | V3        |
| Memory pos. enc.   | True               | V5        |
| Window position    | True               | V7        |
| Sensor value prior | True               | V7        |
| Hierarchical       | False              | V6        |
| Regression head    | False              | V6        |
| Seq. classifier    | False              | V6        |
| Pointer network    | False              | V7        |
| Noise injection    | 0.0                | V6        |

Notably, the learning rate shifted from  $5 \times 10^{-4}$  (optimal at 100–500 epochs in V1–V5) to  $3 \times 10^{-4}$  (optimal at 500+ epochs in V6–V7). The pointer network and drop path regularization show no benefit in V7 and are locked OFF.

## Appendix C.5 Grammar Constraint Analysis

Grammar constraints underwent significant revision between V5 and V6. Initial rules (V5) prohibited command-less sequences, penalizing the common modal-persistence pattern where a G-code line begins directly with a parameter (e.g., X3.291 Z0.0003 under implicit G1). After correcting the type-transition mask to allow BOS→PARAMETER and NUMERIC→COMMAND transitions (Table A4):

**Table A4.** Grammar Constraint Effect (V6, f110\_w256,  $n = 309$ )

|             | Best  | Mean  | Verdict     |
|-------------|-------|-------|-------------|
| Grammar ON  | 79.0% | 54.7% | Top trial   |
| Grammar OFF | 78.1% | 56.3% | Higher mean |

The fixed grammar helps the best-tuned models achieve higher peaks but slightly hurts the average. This is consistent with the observation in [5] that hard constraints can propagate commitment errors: if the model commits to G2, the grammar forces an R prediction that may be wrong, whereas an unconstrained model could recover by not predicting R at all.

## Appendix C.6 V5 Feature Flag Analysis

**Table A5.** Feature Flag Analysis (V5, f110\_w256 encoder,  $n = 117$ )

| Feature            | Best ON | Best OFF | Mean $\Delta$ | $d$   |
|--------------------|---------|----------|---------------|-------|
| MWC + MPE*         | 86.4    | 70.5     | +14.1         | 1.09  |
| Memory pos. enc.   | 86.4    | 70.5     | +6.2          | 0.65  |
| Grammar constraint | 78.1    | 86.4     | +0.0          | 0.00  |
| Hierarchical       | 86.4    | 81.0     | +0.1          | 0.01  |
| Regression head    | 81.0    | 86.4     | −1.3          | −0.13 |

\*MWC  $k=1$  combined with memory positional encoding.

$d$  = Cohen's  $d$  on mean test\_seq across trials.

Key findings: MWC + memory positional encoding is the dominant combination ( $d = 1.09$ ); grammar constraints are neutral on average; hierarchical conditioning is redundant with memory positional encoding; regression heads are slightly harmful (−1.3% mean).

## Appendix C.7 V7 Feature Flag Analysis

**Table A6.** V7 Feature Analysis (157 trials, all folds)

| Feature         | Best ON/OFF | Mean $\Delta$ | Verdict            |
|-----------------|-------------|---------------|--------------------|
| Sensor prior    | 91.4 / 91.4 | −0.7          | Neutral            |
| Pointer network | 91.4 / 91.4 | +0.3          | Neutral            |
| Grammar constr. | 91.4 / 91.4 | +0.9          | Slight positive    |
| Drop path 0.1   | 91.4 / 91.4 | −1.7          | Slight negative    |
| MWC $k=1, 2$    | 91.4 / —    | —             | All trials use MWC |

With 157 completed trials, no V7 feature flag shows a statistically significant effect on best accuracy—all achieve 91.4% regardless of setting. This convergence confirms that the preprocessing fixes dominate: once the correct metadata is available, the decoder achieves strong performance regardless of architectural embellishments.

## Appendix C.8 Killed Features

Two features were tested in V6 and permanently removed:

**Sequence classifier (V6):** A 34-class head predicting the entire G-code sequence as a classification target, intended as a prior for the autoregressive decoder. Effect: severely negative (−9.7%, best 59.1% vs. 79.0% without). The classifier’s global prediction interfered with the decoder’s left-to-right token generation rather than guiding it.

**Noise injection (V6):** Gaussian noise ( $\sigma \in \{0.01, 0.05\}$ ) added to encoder memory as data augmentation. Effect: negative at all levels tested. The frozen encoder already incorporates a denoising reconstruction objective [1]; additional noise in the decoder’s input degraded an already-compressed representation without providing regularization benefit.

## Abbreviations

|              |                                                       |
|--------------|-------------------------------------------------------|
| ANOVA        | Analysis of Variance                                  |
| ASR          | Automatic Speech Recognition                          |
| BOS          | Beginning of Sequence                                 |
| BPE          | Byte Pair Encoding                                    |
| CAM          | Computer-Aided Manufacturing                          |
| CNC          | Computer Numerical Control                            |
| CNN          | Convolutional Neural Network                          |
| DTAE         | Denoising Temporal Attention Encoder                  |
| EOS          | End of Sequence                                       |
| GPU          | Graphics Processing Unit                              |
| HSS          | High-Speed Steel                                      |
| IMU          | Inertial Measurement Unit                             |
| LLM          | Large Language Model                                  |
| LSTM         | Long Short-Term Memory                                |
| MCCDAQ       | Measurement Computing Data Acquisition                |
| MLP          | Multi-Layer Perceptron                                |
| MM-DTAE-LSTM | Multi-Modal Denoising Temporal Attention Encoder–LSTM |
| MWC          | Multi-Window Context                                  |
| MPE          | Memory Positional Encoding                            |
| NIST         | National Institute of Standards and Technology        |
| PDM          | Pulse Density Modulation                              |
| pp           | Percentage Points                                     |
| RMS          | Root Mean Square                                      |
| RPM          | Revolutions Per Minute                                |
| UHMW-PE      | Ultra-High Molecular Weight Polyethylene              |

## References

1. Eacuella, S.S.; Prasad, R.S.; Sodhi, M.S. Multimodal Sensor-Fusion Deep Learning for CNC Toolpath and Condition Classification: A Cross-Validated Ablation Study. *Machines* **2026**. Under review.
2. Teti, R.; Jemielniak, K.; O’Donnell, G.; Dornfeld, D. Advanced monitoring of machining operations. *CIRP Annals* **2010**, *59*, 717–739.
3. Serin, G.; Sener, B.; Ozbayoglu, A.M.; Unver, H.O. Review of tool condition monitoring in machining and opportunities for deep learning. *The International Journal of Advanced Manufacturing Technology* **2020**, *109*, 953–974.
4. Graves, A.; Mohamed, A.r.; Hinton, G. Speech Recognition with Deep Recurrent Neural Networks. *IEEE International Conference on Acoustics, Speech and Signal Processing (ICASSP)*, 2013, pp. 6645–6649.
5. Eacuella, S.S.; Prasad, R.S.; Sodhi, M.S. G-Code Grammar for LLMs: Hierarchical Tokenization of CNC Machine Instructions, 2026. In preparation; tokenizer implementation available at [https://github.com/stephenseacuella/gcode\\_fingerprinting](https://github.com/stephenseacuella/gcode_fingerprinting).
6. Eacuella, S.S.; Prasad, R.S.; Sodhi, M.S. Physical-Layer Anomaly Detection for CNC Machining via Sensor-to-G-Code Reconstruction, 2026. In preparation; companion paper to the present work, validating the anomaly-detection application suggested by the reconstruction error structure.

7. Mohanraj, T.; Shankar, S.; Rajasekar, R.; Sakthivel, N.; Pramanik, A. Tool condition monitoring techniques in milling process — a review. *Journal of Materials Research and Technology* **2020**, *9*, 1032–1042.
8. Zhu, K.; Wong, Y.S.; Hong, G.S. Wavelet analysis of sensor signals for tool condition monitoring: A review and some new results. *International Journal of Machine Tools and Manufacture* **2009**, *49*, 537–553.
9. Bergs, T.; Gierlings, S.; Auerbach, T.; Klink, A.; Schraknepper, D.; Augspurger, T. The Concept of Digital Twin and Digital Shadow in Manufacturing. *Procedia CIRP* **2021**, *101*, 81–84.
10. Stathatos, E.; Tzimas, E.; Benardos, P.; Vosniakos, G.C. Convolutional Neural Networks for Raw Signal Classification in CNC Turning Process Monitoring. *Sensors* **2024**, *24*, 1390.
11. Wang, H.; Wang, S.; Sun, W.; Xiang, J. Multi-Sensor Signal Fusion for Tool Wear Condition Monitoring Using Denoising Transformer Auto-Encoder ResNet. *Journal of Manufacturing Processes* **2024**, *124*, 1227–1238.
12. Weichert, D.; Link, P.; Stoll, A.; Rüping, S.; Ihlenfeldt, S.; Wrobel, S. A review of machine learning for the optimization of production processes. *The International Journal of Advanced Manufacturing Technology* **2019**, *104*, 1889–1902.
13. Sutskever, I.; Vinyals, O.; Le, Q.V. Sequence to Sequence Learning with Neural Networks. *Advances in Neural Information Processing Systems (NeurIPS)*, 2014, Vol. 27.
14. Vaswani, A.; Shazeer, N.; Parmar, N.; Uszkoreit, J.; Jones, L.; Gomez, A.N.; Kaiser, Ł.; Polosukhin, I. Attention is All You Need. *Advances in Neural Information Processing Systems (NeurIPS)*, 2017, Vol. 30.
15. Abdelaal, M.; Lokadjaja, S.; Engert, G. GLLM: Self-Corrective G-Code Generation using Large Language Models with User Feedback. *Proceedings of the 21st Conference on Database Systems for Business, Technology and Web (BTW), Industrial Track*, 2025.
16. Wang, Z.; Jadhav, Y.; Pak, P.; Barati Farimani, A. Image2Gcode: Image-to-G-code Generation for Additive Manufacturing Using Diffusion-Transformer Model. *arXiv preprint arXiv:2511.20636* **2025**.
17. Jeon, J.; Sim, Y.; Lee, H.; Han, C.; Yun, D.; Kim, E.; Nagendra, S.L.; Jun, M.B.; Kim, Y.; Lee, S.W.; Lee, J. ChatCNC: Conversational machine monitoring via large language model and real-time data retrieval augmented generation. *Journal of Manufacturing Systems* **2025**, *79*, 504–514.
18. Jignasu, A.; Marshall, K.; Ganapathysubramanian, B.; Balu, A.; Hegde, C.; Krishnamurthy, A. Towards Foundational AI Models for Additive Manufacturing: Language Models for G-Code Debugging, Manipulation, and Comprehension. *arXiv preprint arXiv:2309.02465* **2023**.
19. Scholak, T.; Schucher, N.; Bahdanau, D. PICARD: Parsing Incrementally for Constrained Auto-Regressive Decoding from Language Models. *Proceedings of EMNLP*, 2021.
20. Willard, B.T.; Louf, R. Efficient Guided Generation for Large Language Models. *arXiv preprint arXiv:2307.09702* **2023**.
21. Dong, Y.; Ruan, C.F.; Cai, Y.; Lai, R.; Xu, Z.; Zhao, Y.; Chen, T. XGrammar: Flexible and Efficient Structured Generation Engine for Large Language Models. *Proceedings of Machine Learning and Systems (MLSys)*, 2025.
22. Zonouz, S.; Rrushi, J.; McLaughlin, S. Detecting Industrial Control Malware Using Automated PLC Code Analytics. *IEEE Symposium on Security and Privacy*, 2014.
23. National Institute of Standards and Technology. Guide to Operational Technology (OT) Security. Technical Report SP 800-82 Rev. 3, NIST, 2023.
24. Vogl, G.W.; others. Research Needs for Cyberphysical Systems in Machining and Machine Tools: FMNet 2024 Report. Technical report, National Institute of Standards and Technology, 2024.
25. Sturm, L.D.; Williams, C.B.; Camelio, J.A.; White, J.; Parker, R. Cyber-physical vulnerabilities in additive manufacturing systems. *International Solid Freeform Fabrication Symposium*, 2014, pp. 951–963.
26. Yampolskiy, M.; King, W.E.; Gatlin, J.; Belikovetsky, S.; Brown, A.; Skjellum, A.; Elovici, Y. Security of Additive Manufacturing: Attack Taxonomy and Survey. *Additive Manufacturing* **2018**, *21*, 431–457.
27. Kimmell, J.C.; Orlyanchik, V.; Sturm, L.; Dawson, J.; Taylor, C. Detecting Control Injection Attacks Using Energy Data Anomalies in Computer Numerical Control Machining. *Critical Infrastructure Protection XVIII* **2025**, *725*, 43–63.
28. Lin, T.Y.; Goyal, P.; Girshick, R.; He, K.; Dollár, P. Focal Loss for Dense Object Detection. *IEEE International Conference on Computer Vision (ICCV)*, 2017, pp. 2980–2988.
29. Bengio, S.; Vinyals, O.; Jaitly, N.; Shazeer, N. Scheduled Sampling for Sequence Prediction with Recurrent Neural Networks. *Advances in Neural Information Processing Systems (NeurIPS)*, 2015, Vol. 28.

1225 © 2026 by the authors. Submitted to *Sensors* for possible open access publication under the terms and conditions  
1226 of the Creative Commons Attribution (CC BY) license (<http://creativecommons.org/licenses/by/4.0/>).